Multi-Project Involvement and Volunteer Developers' Continued Contributions to Open Source Software Projects

Pratyush Nidhi Sharma*, The University of Alabama, (pnsharma@ua.edu)
Hani Safadi, University of Georgia, (hanisaf@uga.edu)
Elena Karahanna, University of Georgia, (ekarah@uga.edu)
John Hulland, University of Georgia, (jhulland@uga.edu)
Munsif Sokiyna, The University of Alabama, (msokiyna@crimson.ua.edu)

Note: This paper is forthcoming in the Journal of Strategic Information Systems.

Multi-Project Involvement and Volunteer Developers' Continued Contributions to Open Source Software Projects

ABSTRACT

Open source software (OSS) ecosystems consist of projects that compete for volunteer developers who divide their attention across projects. Multi-project involvement makes it challenging for a focal project to attract a critical mass of volunteers to make long-term continued code contributions. Adopting a context-specific theorizing approach, we extend established OSS motivation literature to the multi-project context by examining how developers' continued focal project contributions are affected by the relative attractiveness of competing projects with which they are concurrently involved. Specifically, we focus on two well-established extrinsic motivators, developer status and project success, and examine how their *relative* values across a developer's project portfolio influence continued contribution to a focal project. Using data from 2,931 volunteer developers affiliated with 445 focal projects and 4,448 competing projects hosted on Google Code, we find that developers' continued contribution to a focal project is heightened when they have lower relative status in competing projects and when competing projects have lower relative success than the focal project. Asymmetrically, we also find that continued contribution to the focal project is not diminished when developers have higher relative status in competing projects or when those projects are more successful than the focal project. These findings extend work on continued contribution by highlighting the multi-project embedded nature of work performed in OSS ecosystems and have important implications for understanding developer retention across projects.

Keywords: Open Source Software Development, Continued Contribution, Context-Specific Theorizing**,** Motivation, Relative Incentives, Competing Projects, Multi-Project Participation, Pareto-NBD

INTRODUCTION

Open source software (OSS) development relies heavily on volunteer developer participation (Ho and Rai 2017), accounting for more than half of all contributions to successful OSS projects (Riehle et al. 2014). Whereas a major reason for the failure of most OSS projects is the lack of a critical mass of volunteer developers, their ongoing contributions are essential to project survival and success (Weng and Soh 2023). Regenerating teams or shepherding new developers is particularly challenging for OSS projects because of the general absence of formal recruiting, onboarding, and training processes (Crowston et al. 2004; von Krogh et al. 2003) and competition from other projects (David and Rullani 2008). Furthermore, OSS projects are vulnerable to a "contribution barrier," in which newcomers find it difficult to acclimate to the project's complex architecture. Once developers overcome this barrier, it is in the best interest of projects to retain and promote their continued contributions (Fang and Neufeld 2009). This remains a major challenge, and a deeper understanding of associated factors is a critical component of the OSS research agenda (von Krogh et al. 2012).

OSS development occurs in a competitive ecosystem where projects compete for developers' attention, time, and effort (Sutanto et al. 2021; Weng and Soh 2023). The vast majority of OSS developers do not allocate all their time to a single project (Cummings and Haas 2012) but instead work simultaneously on several projects (David and Rullani 2008; Grewal et al. 2006; Krishnamurthy 2006). Thus, their continued contribution to a specific *focal project* cannot be independent of their contributions to other *competing projects* with which they are also concurrently affiliated. We use the term focal project to refer to the specific OSS project whose continued contribution we seek to explain and competing projects to refer to the other projects with which a developer is simultaneously affiliated.[1] A developer's contribution to the focal project is likely influenced by the characteristics of both the focal and competing projects, making it important to jointly consider the influence of these characteristics on developers' continued contributions.

[1] The focal and competing designations are analytically determined for the purpose of this study and do not reflect a permanent or exclusive commitment of the developer. A project that serves as a competing project for one developer may be focal for another, and a developer's primary locus of engagement may shift over time. The operational details of how the focal project was identified in our sample are described in the Empirical Study section.

This understanding is essential for project managers and maintainers seeking to attract and sustain developers' contributions within a large ecosystem of competing OSS projects.

Despite developers often contributing to multiple projects, previous work on OSS continued contribution has focused exclusively on the focal project and its associated factors (the focal-project perspective) while largely ignoring competing project factors (the competing-project perspective). At the individual developer level, prior research has examined developers' motivations to contribute to a focal project (e.g., von Krogh et al. 2012; Roberts et al. 2006; Stewart and Gosain 2006). At the project level, research has examined focal project characteristics that influence developers' motivations for continued contribution (e.g., Ho and Rai 2017). While this body of work has considerably advanced our understanding of continued contribution to a focal project, it remains unclear how competing project factors, at both the individual and project levels, affect focal project contribution.

We extend current understanding of factors influencing continued code contribution by adopting a view that includes characteristics of both the focal project and competing projects, a perspective not used in previous research. We follow a context-specific theorizing approach in which established empirical findings are extended by incorporating factors uniquely salient in the focal context (Hong et al. 2014). The established OSS motivation literature identifies *developer status* (an individual-level factor) and *project success* (a project-level factor) as key extrinsic motivators of continued contribution (Ho and Rai 2017; Roberts et al. 2006; Wu et al. 2007). However, these findings were developed from a focal-project-only perspective. The multi-project feature of OSS ecosystems introduces a distinctive structural context-specific dimension that existing work has not captured - the *relative* nature of these extrinsic motivators across competing projects. Because the extrinsic benefits of status and project success are portable across a developer's professional identity and not tied exclusively to any single project, developers can compare these benefits across their portfolio and allocate their effort accordingly. Thus, we address the following research question: *How do the relative values of developer status and project success across a developer's portfolio of competing projects shape their continued code contribution to a focal project?*

To answer this question, we theorize that the relative nature of these extrinsic motivators operates through two mechanisms. When competing projects offer weaker extrinsic rewards relative to the focal project, the focal project's motivational pull is enhanced (*relative-motivation effect*). Conversely, when competing projects offer stronger extrinsic rewards, developers may redirect their attention toward them at the expense of the focal project (*competitive displacement effect*). We test these hypotheses using data from 2,931 unique developers affiliated with 445 focal projects and 4,448 competing projects hosted on Google Code, spanning a six-month observation window.

Our findings reveal that it is not the absolute but the relative nature of competing projects that influences developers' contributions to a focal project. Specifically, we find that continued contribution to a focal project is heightened when the developer has lower relative status in competing projects and when competing projects have lower relative success than the focal project (a relative-motivation effect). Importantly, we also find an asymmetry. Continued contribution to the focal project is not diminished when the developer has higher relative status in competing projects or when these projects have higher relative success than the focal project (no competitive displacement). This asymmetry suggests that while relative extrinsic motivations attract contributions to the focal project, they do not displace them when competing projects become more attractive. We discuss the theoretical and practical implications of these findings, including what they reveal about developer engagement in multi-project OSS ecosystems.

Our study makes three contributions. First, we extend the OSS continued-contribution literature by shifting from a focal-project-only perspective to a competing-project perspective that considers developers' entire portfolio of project affiliations, providing significantly greater explanatory power. Following Hong et al.'s (2014) approach, we extend established OSS motivation literature by incorporating the relative role of extrinsic motivators as context-specific factors that emerge from the multi-project structure of OSS ecosystems. Second, we document a novel empirical pattern showing the presence of relative motivation effects but the absence of competitive displacement among OSS developers. This deepens our understanding of how developers sustain engagement with focal projects amid competing alternatives. Third, we introduce a new measure of continued code contribution drawn from the Pareto-NBD model

(Schmittlein et al. 1987), that provides a probabilistic, temporally sensitive measure that builds on heuristic approaches used in prior OSS research.

BACKGROUND

The Importance of Continued Code Contribution to OSS Projects

Contributions from volunteer developers are essential for an OSS project's survival and success. These take multiple forms, including code contributions, discussion forum posts, and bug reports. Although many users contribute to supportive and peripheral activities such as documentation and testing, few users undertake active code development tasks (Giuri et al. 2010). Notwithstanding the importance of such supporting activities, code contribution is the primary activity that brings software into existence (Krishnamurthy 2002). Code contributions differ fundamentally from other types of OSS contributions due to the time, effort, prior understanding of the codebase, and expertise required. Indeed, the main contributors to OSS projects are among the top 5% of programmers in terms of skill (Fitzgerald 2005).

The development of OSS projects is less structured than that of traditional software projects. OSS projects often lack a pre-planned roadmap, and developers can contribute on their own schedules (Ho and Rai 2017; Morgan and Finnegan 2014). Because much of the contribution is voluntary, OSS developers usually contribute sporadically. Yet, what matters for project success and sustainability is the *continued code contribution* of volunteer developers (Fang and Neufeld 2009; Spaeth et al. 2014; Xu et al. 2009). Growing the project developer base is important but so is retaining existing developers and promoting their contribution (von Krogh et al. 2003). The contribution barrier makes it difficult for new developers to contribute, given the time and effort required to understand the codebase and complex architecture, especially without formal onboarding processes (Crowston et al. 2004; von Krogh et al. 2003). The survival of OSS projects thus depends on retaining developers who have overcome this barrier by promoting their continued contribution (Fang and Neufeld 2009). Consequently, we focus on continued code contribution as a critical activity leading to OSS projects' growth, sustainability, and success.

Competing-Project Involvement as a Contextual Factor

Modern OSS ecosystems are characterized by many projects competing for developers' attention and expertise. For example, in 2025, GitHub reported more than 630 million total repositories.[2] This abundance of choice means developers' motives to contribute are no longer strongly tied to specific, isolated initiatives; the benefits a developer seeks from contributing to a particular project can also be gained by contributing to others. Thus, to better understand developers' continued code contributions, we use a competing-project perspective, complementing existing work that has traditionally relied on a focal-project perspective. This approach can reveal patterns not apparent in single-project studies (Santos et al. 2013).

Though prior work has theorized a broader view of OSS development (von Krogh et al. 2012), only a few empirical studies have examined multi-project involvement. Concurrent membership in multiple work teams is relatively common in organizations (Wageman et al. 2012) and work on multi-team membership highlights two countervailing forces. On the one hand, positive knowledge spillovers arise when individuals carry valuable knowledge across projects, contributing to individual development and team effectiveness (Sutanto et al. 2021). Working on multiple projects allows developers to tap into different learning opportunities (Singh et al. 2011), apply skills across projects efficiently (Singh and Phelps 2013), balance workloads (O'Leary et al. 2011), and adopt more efficient work practices (Cummings and Haas 2012; Aral et al. 2012).

On the other hand, because the total available time is finite, the time developers can dedicate to a focal project will be reduced when they also engage in competing projects. As the number of competing projects increases, developers may struggle with competing demands that offset efficiencies and decrease their contribution levels (O'Leary et al. 2011). Thus, developers in an OSS ecosystem need to prioritize allocating their time and effort across their project portfolio (David and Rullani 2008; Grewal et al. 2006). While multi-project involvement clearly affects work performance and contribution (Cummings and Haas 2012; Zhu et al. 2014), it remains unclear how competing project characteristics affect continued

[2] GitHub Octoverse, 2025 https://octoverse.github.com/

contribution to a focal project. We seek to complement this literature by examining factors that promote or impede a developer's continued contribution to a focal project vis-à-vis competing projects.

Status and Project Success as Key Extrinsic Motivators of Continued Contribution

OSS developers are motivated by both intrinsic motivations (e.g., enjoyment, learning, community identification) and extrinsic motivations (e.g., status, reputation, economic returns) (Hars and Ou 2002; von Krogh et al. 2012). Developer status and project success have emerged as particularly consequential for continued code contribution (Roberts et al. 2006; Ho and Rai 2017; Wu et al. 2007). These two extrinsic motivators have characteristics that make them especially relevant to the competing-project perspective. First, OSS platforms make status visible through badges, contribution statistics, and formally designated roles (Moon and Sproull 2000; Levina and Arriaga 2014), and project success is signaled through publicly available metrics such as download counts, user adoption, and community size (Crowston et al. 2006; Daniel et al. 2013). Second, status earned in OSS projects enhances a developer's reputation within the broader community and translates into economic returns such as higher wages in paid employment (Hann et al. 2013), career advancement, and skill development opportunities (Singh et al. 2011; Lakhani and Wolf 2005). Similarly, affiliation with successful projects serves as a quality signal to external audiences, including prospective employers (Ho and Rai 2017). Third, and most importantly for our purposes, these properties of observability and portability make status and project success comparable across projects: due to this public visibility, developers can assess their relative standing and the relative success of projects in their portfolio and evaluate the relative extrinsic returns across their project affiliations.

At the individual level, status has emerged as a particularly important extrinsic motivator of continued contribution. Wu et al. (2007) found that developers' motives to enhance their human capital and satisfy their personal needs influenced their intentions to continue contributing. Roberts et al. (2006) tracked code contributions after an initial survey of developers' motives and found that individual developers' extrinsic status motivations are precursors to continued code contribution. Status gained through OSS contribution provides a use value and becomes a goal unto itself (Lakhani and Wolf 2005; Lampel and

Bhalla 2007). Other studies reinforce the centrality of status-related motivators such as feedback and ranking (Hann et al. 2013; Zhang et al. 2013), recognition (Ke and Zhang 2010), and group identity (Bagozzi and Dholakia 2006). The importance of extrinsic motivators such as reputation scores and ranking extends beyond OSS to other online communities. Studying a community Q&A platform, Krishna et al. (2024) found that gains in reputation-based rewards significantly influenced contribution behavior, highlighting the role of visible status in shaping contributor engagement with digital platforms.

At the project level, project success has been identified as a key extrinsic motivator. The success and quality of OSS projects translate into economic returns for participants. Ho and Rai (2017) employ signaling theory and find that accreditation and code acceptance serve as quality signals that complement developer tenure to predict continuance intentions. Contributing to successful projects showcases developers' technical and managerial skills, which can positively affect their wages (Hann et al. 2013). Project success also promotes the recognition and prestige of participating developers within the broader OSS community (Fitzgerald 2005) and is associated with an increased sense of competence among developers who feel responsible for the project's outcomes (Gagné and Deci 2005). These studies suggest that developers' status and project quality signals are significant precursors of continued code contribution.

From Focal-Project to Competing-Project Perspective

Our review demonstrates that status and project success are well-established extrinsic motivators of continued contribution (Roberts et al. 2006; Ho and Rai 2017; Hann et al. 2013). However, this work has been conducted almost exclusively from a focal-project perspective that does not account for a distinctive structural feature of OSS ecosystems: a competitive environment in which developers simultaneously maintain a portfolio of project affiliations. As shown in Table 1 (organized by level of analysis and key antecedents examined), the effect of the relative values of extrinsic motivators across competing projects has not been empirically examined. We address this gap in our theoretical development.

Table 1: Antecedents of OSS Developer Continued Contribution (Focal-Project Perspective)

Study	Setting	Level	Key Antecedents	Operationalization of Continued Contribution
Roberts et al. (2006)	Three Apache projects	Individual	Status motivation, use value, education, experience	Archival: Code submitted and accepted
Hann et al. (2013)	Three mainstream OSS projects	Individual	Economic return from technical and managerial contributions	Code contribution, project management activities
Wu et al. (2007)	SourceForge, Debian, OpenWebmail	Individual	Human capital enhancement, personal needs, career advancement	Survey: Continued participation intention
Ho and Rai (2017)	SourceForge, Freshmeat, other OSS projects	Individual & Project	Tenure, experience, role; project accreditation, code acceptance	Survey: Intention to continue participation
Bagozzi and Dholakia (2006)	Linux user groups	Individual	Perceived behavioral control, attitudes, positive emotions, social identity	Message posting; Survey: Intention to participate
Zhang et al. (2013)	SourceForge projects	Individual & Project	Status, community response	Archival: Long-term continued participation in forums
Fang and Neufeld (2009)	One OSS community	Individual	Peer recognition, community construction, identity	Message posting, bug reporting, code commits; Interview

THEORETICAL DEVELOPMENT

Established frameworks such as Self-Determination Theory (Deci and Ryan 1985) and Motivation Crowding Theory (Frey and Jegen 2001) provide valuable insights into OSS developer motivations, but do not directly address how *relative extrinsic incentives* distributed across competing alternatives shape effort allocation. Extending the established OSS motivation findings to this competing-project context therefore requires an approach suited to deriving context-specific factors from existing knowledge. Hong et al. (2014) argue that richer theoretical understanding can be achieved through context-specific theorizing, an approach in which established findings are extended by incorporating factors uniquely salient in a particular context. The core logic is that context-specific factors can be derived from the intersection of established constructs

with the unique features of the focal context. Rather than borrowing an external theory wholesale, this approach allows researchers to derive context-specific variables grounded in constructs established in the existing literature but formulated to reflect their manifestation in a specific setting.

The OSS motivation literature has consistently shown that status and project success drive continued contribution across multiple studies (Roberts et al. 2006; Ho and Rai 2017; Wu et al. 2007; Hann et al. 2013). These studies provide an established theoretical foundation from which we derive our context-specific extension using the competing-project perspective. Specifically, we focus on the relative values of developer status and project success across a developer's portfolio of competing projects as context-specific factors that the focal-project-only perspective fails to capture.

We propose two sets of contrasting hypotheses that focus on developers' status across competing projects and the success of those projects. Competitive displacement represents a threat to continued contribution to the focal project. When the focal project is perceived as less attractive than competing projects, extrinsic incentives from those projects will spur the developer's attention toward competing projects, resulting in reduced contribution to the focal project (H1a and H2a). In contrast, the relative-motivation effect supports focal project contribution. When the focal project is perceived as more attractive than competing projects, its relative motivational pull may enhance or sustain contribution for the focal project (H1b and H2b). Figure 1 presents our research model, which incorporates both the focal-project perspective established in prior work and the competing-project perspective, the focus of this study.

Effects of Developers' Relative Status in Competing Projects

Prior studies suggest that recognition, reputation, and status are important motivators in sustaining continued contribution. Recognition by other developers is one form of external reward for a developer (Hars and Ou 2002; Lerner and Tirole 2002). Recognition is also a form of feedback on the quality of the developer's work, showing developers that others are using their contributions, satisfying their need to feel competent (von Krogh et al. 2012).

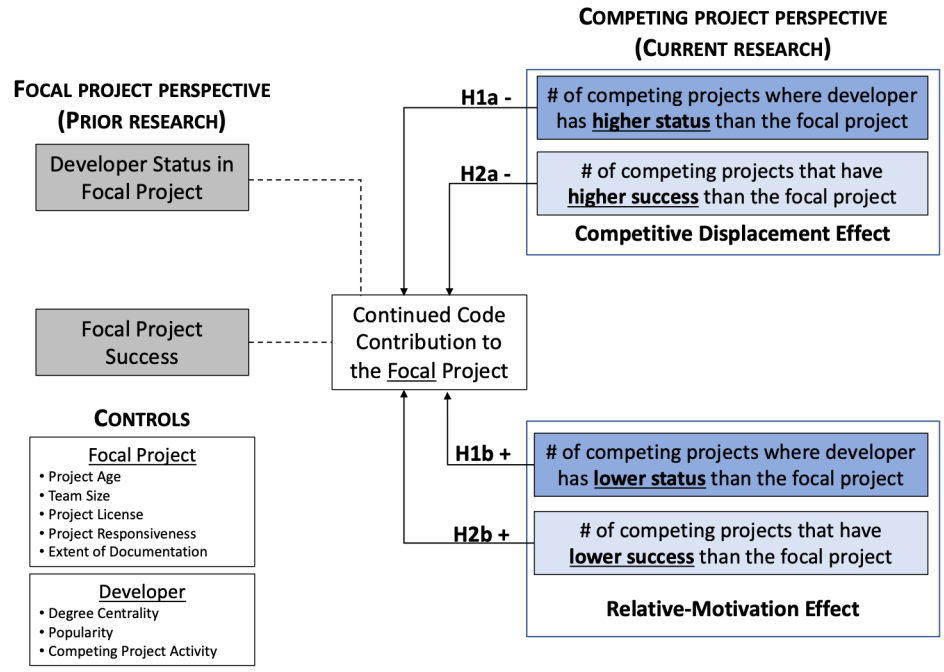


Figure 1: Research Model

Recognition is also self-reinforcing, encouraging the developer to continue contributing (Hars and Ou 2002). Roberts et al. (2006) report that an improvement in a developer's performance ranking is associated with a subsequent increase in their status motivation to make substantive contributions to the project. In fact, they suggest that projects should nurture status motivations by visibly recognizing distinguished developers on their websites. Other studies show that feedback and ranking help sustain developers' participation (Hann et al. 2013; Zhang et al. 2013), and that reputation and recognition for doing good work enhance developers' motivation to contribute (Ke and Zhang 2010). Recognition results in the motivation to continue contributing via the development of affective attachment and group identity (Bagozzi and Dholakia 2006).

OSS projects enable developers' reputations and recognition to be highlighted through visible status indicators such as badges, contribution statistics, and roles that reflect their contributions (Moon and Sproull 2000; Levina and Arriaga 2014). The mechanisms and rules used to endow developers with status markers and roles are negotiated and thus vary across projects and over time (O'Mahony and Ferraro 2007). Large, well-established projects that have typically been the focus of OSS literature generally follow

rigorous rules to accord status to developers (Jensen and Scacchi 2007). For example, Apache and Mozilla endow developers with ownership of modules through their continued contributions to them (Mockus et al. 2002). Other projects use voting mechanisms to confer higher status on developers. In contrast, smaller and less well-known projects often entice new developers by granting them status to secure their commitment without requiring significant prior contributions (Binstock 2011; Lester 2012). Finally, developers can assume higher status as a by-product of creating a new or resurrecting a dead project (Finley 2013).

Status enables developers to compare themselves with one another, be recognized, feel competent, and identify strongly with the project, motivating continued contribution. Regardless of how it is achieved, higher status is self-reinforcing and confers prestige, making developers more committed to contributing to the project (Mockus et al. 2002).

We extend these arguments to theorize how the developer's relative status in competing projects (vis-à-vis the focal project) is associated with continued contribution to the focal project. Multi-project affiliation is a widespread practice in OSS development (Cummings and Haas 2012; Zhu et al. 2014). Developers often work simultaneously on multiple projects, with possibly different statuses in each (David and Rullani 2008; Grewal et al. 2006; Krishnamurthy 2006). The recognition and reputation benefits of membership in a focal project can be substituted by membership in competing projects. Because these benefits depend on the developer's status, they can be more readily obtained through competing projects where the developer has higher status relative to the focal project. Therefore, affiliation with competing projects in which developers have a higher status than in the focal project is expected to result in lower extrinsic motivation to continue contributing to the focal project.

The extrinsic benefits that status confers are portable. When competing projects offer these benefits more readily, a rational allocation of scarce time and effort would favor the projects where the greatest extrinsic return can be obtained, redirecting the developer's attention away from the focal project (*competitive displacement*). However, affiliation with competing projects in which developers have lower status than in the focal project cannot substitute for the status benefits derived from the focal project. This

would highlight the salience of maintaining and curating their higher status in the focal project by increasing their contribution to it and continuing to extract the associated benefits (*relative-motivation effect*).

Therefore, in a portfolio of competing projects, holding constant the developer's non-status-related attachments to each project, a developer will focus on the project(s) in which he/she has the highest status. Given the limits on the time and effort a developer can devote to projects, as the number of competing projects in which they have higher status relative to the focal project increases, their motivation to continue contributing to the focal project will be impeded. Conversely, as the number of competing projects in which a developer has a lower status relative to the focal project increases, their motivation to continue contributing to the focal project will be enhanced.

H1: A developer's relative status in competing projects affects contribution to the focal project such that:

Competitive displacement effect *(a): The number of competing projects in which a developer has a higher status relative to the focal project is negatively associated with continued code contribution to the focal project.*

Relative-motivation effect *(b): The number of competing projects in which a developer has a lower status relative to the focal project is positively associated with continued code contribution to the focal project.*

Effects of Relative Project Success of Competing Projects

To explain long-term participation in OSS projects, von Krogh et al. (2012) suggest that OSS development is a social practice in which institutional factors affect developers' motivations and that examining individual motivation in isolation is insufficient. Consistent with this view, we examine project success as a project-level factor that shapes the motivational landscape for individual developers. In addition to individual status benefits, many OSS developers seek to derive future economic benefits from their contributions to projects (Lakhani and Wolf 2005). Even when unpaid, developers can still expect economic returns from their contributions through learning, resume building, and skill development (Singh et al.

2011). Indeed, whereas economic incentives were attenuated or missing in early OSS projects (von Hippel and von Krogh 2003), more recent evidence suggests that OSS developers are now extracting greater economic returns (Hann et al. 2013).

The success of a project is an important factor in determining the value developers can extract from associating with it. The success of the focal project provides a strong extrinsic incentive to sustain developers' continued contribution to it. Building on signaling theory, Ho and Rai (2017) find that the quality of OSS projects, reflected in the processes used to maintain input quality (i.e., developer quality) and output quality (i.e., code quality), leads to sustained volunteer contributions. Similarly, Hann et al. (2013) report that developers' technical and managerial skills gained through voluntary contributions to successful OSS projects were associated with higher pay outside of these projects, highlighting the importance of project success in garnering external recognition.

In addition to economic benefits, the project's success promotes the recognition and prestige of its developers within the OSS community at large. For example, developers of the Linux kernel are all well-recognized for their technical wizardry (Fitzgerald 2005). Becoming a member of attractive and technically esteemed projects captures developers' interest and attention (Snyder et al. 1986). Furthermore, the success of a project is associated with an increased sense of competence among its developers, who feel responsible for the project's success. This increased competence is associated with motivation to contribute (Gagné and Deci 2005). In contrast, engagement in failing projects threatens developers' competence and leads to reduced self-esteem and diminished contributions. The success of the focal project also serves as a status signal that motivates developers to continue contributing.

We build on these arguments to theorize how the relative success of competing projects (vis-à-vis the focal project) is associated with continued contribution to the focal project. As noted, developers expect to gain reputational and economic benefits from affiliation with successful OSS projects (Ho and Rai 2017; Lakhani and Wolf 2005). Thus, the benefits of membership in a focal project can be substituted by membership in more successful competing projects. Competing projects provide additional avenues for developers to demonstrate and nurture their technical competence, be recognized for their work, and attain

economic rewards. Developers can extract the greatest benefit by maintaining, highlighting, and curating their affiliation with the most successful projects in their portfolio. Thus, how developers allocate their effort and motivation to contribute to a focal project will depend on how it compares with the success of competing projects in their portfolio.

Holding constant the developer's non-success-related attachments to each project, extrinsic incentives such as reputation and economic benefits can be better realized by being affiliated with competing projects that are more successful than the focal project. Because the economic and reputational returns of project success are transferable across a developer's career and not confined to the focal project, developers affiliated with more successful competing projects may redirect their effort toward those projects to maximize these portable returns (competitive displacement). In contrast, their involvement in competing projects with lower success relative to the focal project will motivate them to preferentially maintain, highlight, and curate their affiliation with the more successful focal project to optimize benefit extraction (relative-motivation effect). Therefore, we propose that:

H2: The relative success of competing projects affects contribution to the focal project such that:

Competitive displacement effect *(a): The number of competing projects with higher success relative to the focal project is negatively associated with continued code contribution to the focal project.*

Relative-motivation effect *(b): The number of competing projects with lower success relative to the focal project is positively associated with continued code contribution to the focal project.*

EMPIRICAL STUDY

Data and sample

Data were collected from Google Code, an online OSS forge that provided a bundle of software development tools, including communication, version control, and source code repositories. It was among

the largest and most popular OSS platforms[3] and allowed access to source code repositories, bug tracking, documentation, Google groups, releases, downloads, and project statistics. It also allowed project developers to be listed under one of three status categories (in descending order of status): owner, committer, or contributor (Google 2015). Importantly, Google Code enforced a clear delineation of roles and responsibilities across the three status levels. All three statuses allowed developers to contribute code, which enables us to examine the theorized motivational effects of developer status. However, higher status conferred more organizational privileges and responsibilities on developers. Owners held the highest privileges, allowing them to change the project structure, code, and workforce (i.e., they could move people in or out of the project). Committers and contributors could contribute code but not manage the larger project structure or the workforce. However, code contributions by contributors were subjected to formal review before being incorporated into the codebase (Hanselman 2016). These distinctive features made Google Code an ideal platform to test our hypotheses.

In contrast to prior work on code contribution that mainly examined selected, large, and well-established OSS projects, we sampled OSS projects of various sizes and ages. We used a five-step process to create our study sample to assess the competing project perspective. First, we randomly sampled 5,500 project repositories on Google Code. A total of 13,989 unique developers contributed to these projects. We downloaded available information from the project web pages (e.g., project description, number of developers, project license, wiki documentation, issues, and number of downloads) as well as the information related to the developers working on them (e.g., status in projects, stars, project association).

Second, we collected all code commit activity across the projects by downloading the project repositories to a local machine and using the CVSAnalY tool to extract code commit data.[4] To create our dependent variable (i.e., developers' probability of continued code contribution), we collected information

[3] Launched in 2002 as a free forge for open source projects, Google Code was one of the largest open source repositories historically and hosted up to 1.4 Million projects by 2016. https://code.google.com/. FLOSS mole also provides archival access to projects hosted in Google Code and other forges. http://flossmole.org/content/getting-data

[4] https://github.com/MetricsGrimoire/CVSAnalY

regarding individual developers' code commit activity in a 180-day observation window starting from the time when predictor variables were collected in May 2012. This information included the author of the code, the number of code commits ("frequency"), and the timestamps of code commits to assess the "recency" of their activity. Both frequency and recency are used as inputs to generate the probability that a developer is "alive", i.e., active at the end of the observation period.

Third, following previous research (Daniel et al. 2013; Setia et al. 2012; Stewart et al. 2006), we excluded the following projects: (1) all projects with one developer (since involvement in such a project is not meaningful in a team context), (2) projects not based on SVN control,[5] (3) projects that were moved to different project hosting sites (e.g., GitHub, SourceForge), (4) projects that were academic course or class-related and hence potentially temporary, hosted only documents and not code, or were test projects created by developers to explore the Google code environment,[6] (5) projects sponsored by a company such as Google (to ensure that developers worked on a purely volunteer basis), and (6) projects that showed no code commit activity in the one month period before data collection (i.e., possible abandonware). The resulting final sample is comprised of 445 focal projects and 2,931 unique developers.

Fourth, noting that the developers in our final sample worked on another 4,448 "competing" projects, we complemented our dataset with additional information regarding the competing projects the developers were associated with, their status in each such project, and their code commit activities in these projects, using the first two steps described above. In summary, our sample comprises data on 2,931 volunteer developers affiliated with 445 focal projects and simultaneously associated with 4,448 competing projects. Thus, our sample is fairly sizable from both the developer- and project-level perspectives. On average, developers were associated with 3.26 projects (std. dev. = 5.10), and each focal project had, on

[5] About 82% of the core projects in our sample used SVN, while 11% used Mercurial and 7% used GIT systems. Due to the differences in the way workflow is organized in these source code management systems, Rodriguez-Bustos and Aponte (2012) found differences among developer contributions when projects migrated from one system to the other. Therefore, in order to control for any differences among projects that use SVN versus GIT or Mercurial, we excluded projects that used Mercurial or GIT repositories.

[6] Care was taken to manually scan the project descriptions.

average, 6.79 developers (std. dev. = 7.37). Table 2 summarizes the operationalization of constructs of interest and control variables in our model.

Table 2: Constructs & Measurements

Variable Level	Construct	Operationalization	Variable Name	Type
FOCAL PROJECT				
Developer	Continued Code Contribution to Focal Project	Pareto-NBD probability of continued code contribution at the end of the 180 day period in the focal project.	*p(Alive)*	Dependent
	Popularity	Log of number of stars received by the developer.	*Stars*	Control
	Degree Centrality	Number of other unique developers a developer works with across projects.	*Centrality*	Control
	Status	Developer's listed status on the focal project. Owner is the base comparison group.	*Committer*	Control
			Contributor	
Focal Project	Age	Number of years since the first code commit in the focal project.	*Age*	Control
	License Type	License listed by the focal project.	*PermissiveLicense*	Control
			OtherLicense	Control
	Team Size	Number of developers working on the focal project.	*NumDevelopers*	Control
	Extent of Documentation	Number of wiki pages containing focal project documentation.	*WikiPages*	Control
	Responsiveness	Proportion of issues resolved in focal project.	*ProportionIssuesResolved*	Control
	Success	Log of number of times focal project has been downloaded.	*Downloads*	Control
COMPETING PROJECTS				
Competing	Concurrent	Number of	*CompetingActivity*	Control

Projects	Competing Project Activity	concurrent code commits in competing projects.		
	Relative Status in Competing Projects	Number of competing projects where developer has higher status than the focal project.	*HigherStatus*	Predictor
		Number of competing projects where developer has lower status than the focal project.	*LowerStatus*	
	Relative Success of Competing Projects	Number of competing projects that have been downloaded one standard deviation or more number of times than the focal project.	*MoreDownloads*	Predictor
		Number of competing projects that have been downloaded one standard deviation or less number of times than the focal project.	*LessDownloads*	

Description of Measures

Focal-Project Perspective

***Developer's continued code contribution in focal project (dependent variable)*:** We measure a developers' code contribution to the focal project by matching their code commit authorship in the project repository. While code commits are the most straightforward way to assess actual software development (Fitzgerald 2006; Marsan et al. 2012), only a few studies have examined code commits when measuring contribution (Fang and Neufeld 2009; Roberts et al. 2006). Measuring continued contribution in the voluntary OSS setting is a non-trivial problem because code commits are typically dispersed over time, and developers may go inactive for extended periods only to return later. Furthermore, selecting an arbitrary time window will result in right-censored measurements, as a developer can always return to a project after the observation period has ended (e.g., Joyce and Kraut 2006, Zhang et al. 2013). Therefore, simply counting

code commits as a measure of contribution is not enough. Further, developers with more recent contributions are more likely to be active currently. In contrast, developers with heavy contributions in the distant past may have recently become inactive, necessitating an approach that considers the recency of code contributions as well as their frequency.

A contribution of the current study is to resolve this issue by using a probability-based measure of continued contribution, rather than a count (or dichotomous) variable. We use a Pareto (exponential-gamma mixture) timing model and an NBD (Poisson-gamma mixture) activity model to explicitly address right-censored data by predicting the probability of continued contribution (Schmittlein et al. 1987).[7] Online Appendix A provides details of this approach. In short, the probability of continued contribution for each developer at any given point in time (i.e., *p(Alive)*) is estimated based on the extent and timing of their historical contributions to the focal project. This approach accounts for developers' historical activity patterns (i.e., frequency and recency) by adjusting their probability values accordingly.

An additional benefit of taking both frequency and recency into account is the attenuation of potential endogeneity bias (see Online Appendix B for various approaches we used to mitigate it), whereby the frequency of past contributions leads to higher status (and vice versa). This is because, to be evaluated as active, developers with histories of frequent contributions have to not only maintain their contribution levels but also remain active regularly throughout the event window (i.e., show recency). Historically lethargic (less frequent) contributors may also attain a higher predicted probability of being alive (active), based on recent activity and the assumption of a slower individual activity rate, regardless of their status. In contrast, developers with heavy contributions in the distant past (and possibly higher status) but who have been inactive recently will be assigned a low probability of being alive. Thus, taking recency into account and adjusting probabilities accordingly helps attenuate the bias.

Individual Level Predictor and Controls

[7] We also analyzed our data using the raw count of code commits over a 180-day period as an alternate dependent measure, used in past research (e.g., Joyce and Kraut 2006). Our results are robust (see Model I in Online Appendix table B2). Additionally, we also used an alternative 90-day observation period as a robustness check (Model G).

Developer status (control): We use two dummy variables (*Committer* and *Contributor*) to capture a developer's status level in the focal project, with *Owner* as the reference group.[8]

Developer popularity (control): Google Code allows developers to be "starred" and followed by others, which serves as a measure of an individual developer's popularity or community status. Starred developers are likely to be more productive than others, therefore, we use the log of stars received (*Stars)*, as a developer-level control.

Developer centrality (control): A related, community-based network measure focuses on the number of interpersonal ties a developer has through their working relationships with others. Such community networks often influence developer productivity by providing them timely access to quantity and variety of information (Singh et al. 2010). For each developer in our sample, we calculate the number of unique other developers they work with across their entire project portfolio (*Centrality*).

Focal Project Level Controls

***Project Success (control)*:** Defining open-source project success using traditional software success measures, such as revenue generated, on-time completion, or meeting budgetary constraints, is not appropriate (Crowston et al. 2006; Stewart et al. 2006). Accordingly, past research relied on capturing open-source project success as perceived by its users (Daniel et al. 2013; Grewal et al. 2006). A popular measure of a project's success is the number of times its code has been downloaded, which signals its reach and the level of acceptance within the developer and user community (Crowston et al. 2006; Daniel et al. 2013; Setia et al. 2012). Thus, we use the log of the number of times a focal project's code has been downloaded (*Downloads*) as the measure of focal project success.

Project Age (control): Newer projects may suffer from a *liability of newness*, resulting in lower credibility within the development community (Chengalur-Smith et al. 2010), finding it difficult to attract and retain

[8] The average number of code contributions made in the 180-day observation period by owners was 5.73; by Committers 1.70; and by Contributors 0.56. The variability in the developers' statuses across projects is showcased by the relatively high standard deviations for: developers' status as owners (mean = 2, std. dev = 3.88), as committers (mean = 1.11, std. dev = 2.30), and as contributors (mean = 0.14, std. dev = 0.44).

developers. Alternatively, new projects may show high activity due to the excitement they generate among developers. Thus, we controlled for the elapsed time (in years) from the first code contribution made on the focal project (*ProjectAge*).

Project License (control): License restrictiveness can impact developer interest in a focal project (Stewart et al. 2006). Restrictive licenses (i.e., with a *viral clause*, such as the GNU GPL) may affect developers' cost-benefit perceptions of development and use by restricting code compatibility with other software products and commercialization potential. Thus, we distinguish between projects with and without a license carrying a viral clause. Some projects (n=16) did not explicitly list the license being used. We created two dummy variables (*PermissiveLicense*, *OtherLicense*) with restrictive licenses as the reference group.

Project Team Size (control): Butler (2001) has shown that an online community's size affects member dropout rates. Thus, we control for the number of developers of the focal project (*NumDevelopers*).

Project Extent of Documentation (control): Documentation related to a focal project's timeline and its code development is crucial for its users and developers as a knowledge repository. It not only facilitates coordination and collaboration among existing developers but also helps attract new contributors, thereby supporting its sustainability (Aberdour 2007). For example, developer participation increased only after Mozilla introduced extensive documentation (Mockus et al. 2002). Because Google Code utilized wikis for project documentation and knowledge management, we control for the extent of project documentation by aggregating the number of wiki pages available (*WikiPages*).

Project Responsiveness (control): Google Code allowed developers and users to report issues with the project software, including errors, bug fixes, and feature requests, which were then resolved by the developers. We include focal project responsiveness as measured by the proportion of issues resolved (i.e., issues closed divided by issues opened) by the project community, *ProportionIssuesResolved*.

Competing-Project Perspective

Developer relative status in competing projects (individual-level predictor): Because a developer may be involved in multiple projects simultaneously, we created two complementary summary variables to capture

a developer's relative status across competing projects. *HigherStatus* represents the number of competing projects where a developer has a higher status than the focal project, while *LowerStatus* is the number of competing projects where a developer has a lower status than the focal project.[9]

Relative success of competing projects (project-level predictor): There is no clear-cut distinction between when a project would be perceived by developers as relatively more (or less) successful than another in terms of the number of downloads. While a project that has never been downloaded can be considered less successful than one that has been downloaded at least a few times (which signals its acceptance by the community), the distinction becomes less clear for projects that have been downloaded many times. Therefore, we use two complementary summary variables to capture the relative success of competing projects in the developer's portfolio: *MoreDownloads* represents the number of competing projects that have been downloaded one standard deviation or more number of times than the focal project, while *LessDownloads* is the number of competing projects that have been downloaded one standard deviation or less number of times than the focal project.[10] To ensure that our results are not affected by this empirical choice, we conducted several robustness checks using alternative operationalizations of relative success with smaller and larger cutoffs (see Online Appendix B).

Developer concurrent activity in competing projects (individual-level control): We include aggregated concurrent code contribution activity (i.e., the count of code commits) in competing projects during the 180-day observation period, allowing us to control for their activity in competing projects within that period (*CompetingProjectActivity*). Table 3 reports the descriptive statistics.

Table 3: Descriptive Statistics

Variable Level	Variable	N	Mean	SD	Min.	Max.
Focal Project						
Developer	*p(Alive)*	2931	0.17	0.27	0	1
	Stars	2931	1.62	0.91	0	4.1
	Centrality	2931	31.21	43.6	2	823

[9] We also provide evidence of the robustness of our results by incorporating the number of competing projects where a developer has *equal* status as the focal project in Online Appendix B.

[10] The standard deviation of project success variable (log number of downloads, i.e., *Downloads*) was 1.56, which translates to a raw count of 36 downloads. Thus, competing projects that had 36 or more (less) downloads than the focal project were considered relatively more (or less) successful, while projects that fell inside this range were considered equally successful and not counted toward *MoreDownloads* and *LessDownloads.*

	Committer	2931	0.55	0.5	0	1
	Contributor	2931	0.07	0.26	0	1
Focal Project	*Age*	445	2.98	1.98	0.03	15.51
	PermissiveLicense	445	0.51	0.5	0	1
	OtherLicense	445	0.04	0.19	0	1
	NumDevelopers	445	6.8	7.39	2	51
	WikiPages	445	9.69	23.74	0	347
	ProportionIssuesResolved	445	0.53	0.34	0	1
	Downloads	445	3.25	1.56	0	7.15
Competing Projects						
Competing Projects	*CompetingProjectActivity*	2931	2.28	15.78	0	595
	HigherStatus	2931	0.66	1.96	0	35
	LowerStatus	2931	0.26	1.38	0	57
	MoreDownloads	2931	0.35	1.11	0	15
	LessDownloads	2931	1.54	4.26	0	143

ANALYSIS AND RESULTS

In the OSS context, developers are nested within projects (Sharma et al. 2022). Thus, we utilize Hierarchical Linear Modeling (HLM), which is designed to account for nested data and provides robust standard errors by explicitly taking clustering (nesting) into account (Raudenbush and Bryk 2002). More specifically, in our context, the effect of developer characteristics is nested within the effect of focal project-level characteristics. Tables 4 and 5 report the correlations among level 1 (developer) and level 2 (project) variables, respectively.

Table 4: Level 1 (Developer) Correlations

	(1)	(2)	(3)	(4)	(5)	(6)	(7)	(8)	(9)
(1) p(Alive)									
(2) Star	0.026								
(3) Centrality	-0.017	.336**							
(4) Committer	-.161**	.068**	-0.009						
(5) Contributor	-.100**	-0.020	-.044*	-.313**					
(6) CompetingProjectActivity	.123**	0.016	.118**	-.082**	-0.031				
(7) HigherStatus	-.060**	.143**	.261**	.240**	0.021	0.020			
(8) LowerStatus	.039*	.129**	.540**	-.166**	-.053**	.083**	-.038*		
(9) MoreDownloads	0.011	.225**	.550**	-.121**	-.044*	.171**	.162**	.236**	
(10) LessDownloads	0.039*	.148**	.559**	-.059**	-.067**	.149**	.421**	.638**	.211**

Table 5: Level 2 (Project) Correlations

	(1)	(2)	(3)	(4)	(5)	(6)
(1) Age						
(2) PermissiveLicense	0.032					
(3) OtherLicense	-.136**	-.196**				
(4) NumDevelopers	.211**	0.022	0.087			

(5) WikiPages	0.065	-0.041	0.055	.261**		
(6) ProportionIssuesResolved	.210**	0.041	-0.031	.198**	.104*	
(7) Downloads	.358**	-.107*	-.130**	-0.030	0.073	.178**

We start by estimating the unconditional (or null) model, then incrementally build conditional models with predictors at both levels to improve overall model fit. A null model includes no predictors at either level and partitions the variance in *p(Alive)* into variance attributable to level-1 (developer) and level-2 (project) effects. If the latter variance is not significant, HLM is not required. We followed the recommended practice and centered all variables (except dummies) before analysis. All developer-level variables were group-mean centered, while the project-level variables were grand-mean centered to reduce multicollinearity concerns (Raudenbush and Bryk 2002). The fully specified hierarchical model with random effects for developer *"i"* working on project *"j"* (reported as model 2 in Results section) is shown below:[11]

Level-1 (Developer):

$$p(Alive)_{ij} = \beta_{0j} + \beta_{1j}\,(Committer) + \beta_{2j}\,(Contributor) + \beta_{3j}\,(Centrality) + \beta_{4j}\,(Stars) + \beta_{5j}\,(CompetingProjectActivity) + \beta_{6j}\,(HigherStatus) + \beta_{7j}\,(LowerStatus) + \beta_{8j}\,(MoreDownloads) + \beta_{9j}\,(LessDownloads) + r_{ij}$$

Level-2 (Focal Project):

$$\beta_{0j} = \gamma_{00} + \gamma_{01}\,(Age) + \gamma_{02}\,(PermissiveLicense) + \gamma_{03}\,(OtherLicense) + \gamma_{04}\,(NumDevelopers) + \gamma_{05}\,(WikiPages) + \gamma_{06}\,(ProportionIssuesResolved) + \gamma_{07}\,(Downloads) + u_{0j}$$

$$\beta_{1j} = \gamma_{10}$$

$$\beta_{2j} = \gamma_{20}$$

$$\beta_{qj} = \gamma_{q0} + u_q \qquad for\ q = 3 \ldots 9$$

Results

Table 6 provides the full-maximum likelihood estimates for the null model with no predictors at either level (column 1). The intercept is significant ($\gamma_{00} = .178,\ p < .01$) and represents the population-average probability of continued participation across all developers at the end of the 180-day observation

[11] The *Committer* and *Contributor* slopes (β_{1j} and β_{2j}) were "fixed" because their variance components were not statistically significant (Raudenbush and Bryk 2002; Snijders and Bosker 2012), suggesting that the effect of developer status does not vary across projects.

period. The low probability value underscores the significant issue of developer discontinuation in the OSS context. The null model allows variance in *p(Alive)* to be partitioned into level-1 (r_{ij} = *.06614*) and level-2 (u_{oj} = *.00860*) components, with the latter estimate differing significantly from zero ($p < .01$). This indicates that significant variation exists at the project-level, thus requiring a multi-level approach for estimation. The intra-class correlation coefficient (ICC) measures the proportion of variance at level-2 (Raudenbush and Bryk 2002) and indicates that a significant proportion of variance is at the project level (11.5% of the total (*.00860 / (.00860 + .06614)*)).

We next estimated a model representing the focal project perspective (model 1) by including developers' status in the focal project and focal project success. The comparison with the null model indicates that model 1 provides a significantly better fit ($\Delta\chi 2 = 343.229, df = 16, p < .01$). Model 2 extends the focal project perspective by introducing competing project predictors and control variables. The deviance test reports significant improvement in fit for model 2 over model 1 ($\Delta\chi 2 = 76.835, df = 35, p < .01$). AIC also achieves minima for Model 2, which suggests that it is the best among the cohort. Overall, these tests indicate that the competing projects perspective provides a much stronger explanation of developers' continued contributions to focal projects than the focal project perspective alone. Thus, we focus on Model 2 to test our hypotheses.

Table 6: HLM Results for Continued Code Contribution *(p(Alive))*

Level of Analysis	**Fixed Effects**	**Model 0: Null Model**	**Model 1: Focal Project Perspective**	**Model 2: Competing Projects Perspective**
	Intercept (γ_{00})	0.178*** (0.006)	0.263*** (0.014)	0.255*** (0.014)
Controls				
Developer Controls	*Committer (β_1)*		-0.139*** (0.015)	-0.123*** (0.016)
	Contributor (β_2)		-0.190*** (0.018)	-0.170*** (0.018)
	Centrality (β_3)		-0.000 (0.000)	-0.000 (0.000)
	Stars (β_4)		0.032* (0.018)	0.004 (0.017)
	Age (γ_{01})		-0.005 (0.003)	-0.004 (0.003)

Focal Project Controls	*PermissiveLicense* (γ_{02})		-0.026* (0.014)	-0.029** (0.013)
	OtherLicense (γ_{03})		-0.002 (0.046)	0.025 (0.046)
	NumDevelopers (γ_{04})		0.001 (0.000)	0.001 (0.000)
	WikiPages (γ_{05})		0.000 (0.000)	0.000 (0.000)
	Downloads (γ_{07})		0.013** (0.004)	0.010** (0.004)
	ProportionIssuesResolved (γ_{06})		0.065*** (0.022)	0.045*** (0.021)
Competing Project Controls	*CompetingProjectActivity* (β_5)		-	0.001** (0.000)
Hypothesized Effects				
Competing Project Predictors	*HigherStatus* (β_6)		-	-0.001 (0.004)
	LowerStatus (β_7)		-	0.029*** (0.009)
	MoreDownloads (β_8)		-	-0.005 (0.007)
	LessDownloads (β_9)		-	0.005** (0.002)
Model Fit				
Deviance (-2 log likelihood)		599.467	256.238	179.403
Estimated Parameters		3	19	54
Deviance Difference		-	343.229**	76.835**
AIC		605.467	294.238	287.403

†† Comparison group: Owner; ***p < 0.01, **p < 0.05, *p < 0.1; Robust Standard Errors in parentheses.

The intercept γ_{00} is significant and positive ($\gamma_{00} = .255,\ p < .01$) and is interpreted as the average value of *p(Alive)* holding the *Committer* and *Contributor* dummies at zero and other regressors at their means. Thus, the probability of continued contribution after the 180-day period for a *typical* owner is only 25.5%, suggesting that discontinued contribution is a significant problem even among project owners.

Effect of developers' status on their continued code contribution to the focal project

Focal Project Perspective (not hypothesized): We first discuss the effect of a developer status in the focal project on continued code contribution. The dummy variables *Committer* ($\beta_1 = -.123,\ p < .01$) and *Contributor* ($\beta_2 = -.170,\ p < .01$), which capture the differences in *p(Alive)* for committers and contributors vis-à-vis owners, are significant and negative. Thus, a *typical* committer (*p(Alive) = 13.2%*) or contributor

(*p(Alive) = 8.5%*) is *less* likely than an owner to make continued contributions after the 180-day period. These results are in line with previous research from a focal project perspective.

Competing Projects Perspective (hypothesized): Regarding the hypothesized effects (H1a-b), we found that the number of competing projects where a developer has higher status than the focal project, *HigherStatus*, was negative but not significant (β_6 *=-0.001, n.s.*). Thus, H1(a) was not supported. On the other hand, the number of competing projects where a developer has lower status than the focal project, *LowerStatus*, had a significant positive effect on continued code contribution (β_7 *=0.029, p < .01*). This supports H1(b). These findings suggest that being affiliated with competing projects where a developer has relatively lower status than the focal project has an uplifting effect on the developer's code contribution to the focal project (relative-motivation effect), but being affiliated with projects where they have higher status has no negative effect (no competitive displacement).

Effect of project success on developers' continued code contribution to the focal project

Focal Project Perspective (not hypothesized): The success of the focal project in terms of number of downloads *(Downloads)* has a significant positive effect on the probability of continued code contribution (γ_{07} *=0.010, p < .01*).[12] Focal project success in terms of the number of issues resolved (*ProportionIssuesResolved*) also has a significant positive effect (γ_{06} *=0.045, p < .01*). These results are in line with previous research based on the focal project perspective.

***Competing Projects Perspective (hypothesized)*:** Regarding the hypothesized effects (H2a-b), the effect of the number of competing projects that were more successful than the focal project, *MoreDownloads*, was negative but not significant (β_8 *=-0.005, n.s.*). Thus, H2(a) was not supported. On the other hand, the effect of the number of competing projects that were less successful than the focal project, *LessDownloads*, had a significant positive effect on a developer's continued code contribution to the focal project (β_9 *=0.005, p < .05*). This supports H2(b). These findings suggest affiliation with competing projects that have lower success than the focal project propels developers to contribute more to

[12] As noted, project success was assessed ***prior*** to the 180-day code contribution observation window. Thus, these results show the unidirectional effect of project success on subsequent code contribution activity, and not vice-versa.

the focal project (relative-motivation effect), whereas affiliation with projects with higher success does not negatively affect their contributions (no competitive displacement). Thus, in both sets of hypotheses (H1 and H2), we found evidence of the relative-motivation effect but none of competitive displacement. Overall, the competing projects perspective explains developers' continued code contribution above and beyond the focal project perspective alone.

Robustness Checks

To assess robustness to alternative explanations due to outliers, endogeneity bias, alternative construct operationalizations, an alternative dependent variable for continued contribution, additional developer, focal and competing project factors, as well as alternative model specifications, we conducted an exhaustive set of robustness checks. The results, summarized in Table 7 and detailed in the Online Appendix, offer strong evidence of robustness.

Table 7: Summary of Robustness Checks

Type	Description	Hypothesized Effects
Endogeneity Bias	Approaches to mitigate Endogeneity Bias	Consistent
Alternative operationalizations	*Developers' relative status in competing projects:* • Also Include *EqualStatus* in Competing Projects	Consistent
	Relative success of competing projects as compared to focal project: • Strictly lower/higher number of downloads • Difference of 10 or more Downloads • Difference of 100 or more Downloads	Consistent
Inclusion of Additional Control Variables	*Additional Focal project variables:* • Release Frequency • Quality of Documentation	Consistent
	Additional Competing project variables: • Average Age of Competing Projects • Average Proportion of Issues Resolved	
Alternative Model Specifications	Restricted Maximum Likelihood Estimation	Consistent
	90-day Observation Period	*LowerStatus* is not statistically significant
	Sample Excluding Contributors	Consistent
	Alternate Dependent Variable: Count of Code Commits	Consistent
	Outlier Analysis: Winsorized Variables	Consistent
	Interaction between Relative Status in Competing Projects and Relative Success of Competing Projects	Consistent

DISCUSSION AND IMPLICATIONS

Modern OSS projects compete for the attention of volunteer developers who are often concurrently affiliated with multiple other projects. The OSS literature shows that developers are motivated by the benefits of their individual status in projects and by project success (Ho and Rai 2017; Roberts et al. 2006; Wu et al. 2007). However, developers' code contributions to a specific focal project cannot be considered in a vacuum, as has been the norm in past OSS research, but should be examined holistically, accounting for the developer's entire project portfolio. Existing motivation research in OSS (reviewed in Table 1) has focused on explaining developers' behavior within a single focal project and has not addressed how the relative values of developer status and project success across competing projects shape continued contribution to a focal project. Established frameworks such as Self-Determination Theory (Deci and Ryan 1985) and Motivation Crowding Theory (Frey and Jegen 2001) provide valuable insights into developer motivations within a given context, but they do not directly theorize how developers compare and respond to relative extrinsic incentives distributed across a portfolio of competing projects. Because no existing framework directly encompasses this competing-project dimension, we adopted a context-specific theorizing approach (Hong et al. 2014), extending established motivation findings in OSS to the competing-project perspective by introducing the relative values of status and project success across competing projects as context-specific factors that the focal-project-only perspective has not captured. Examining how the relative values of these two extrinsic motivators across a developer's portfolio of competing projects shape their continued code contribution to a focal project, we found that it is the relative, rather than the absolute, nature of competing projects that matters.

Our findings corroborate prior OSS work at the focal-project level, calling for attention to both individual- and project-level factors when examining contribution to OSS projects (von Krogh et al. 2012): higher status within a focal project leads to an increased likelihood of developers' continued contribution (Mockus et al. 2002; O'Mahony and Ferraro 2007; Roberts et al. 2006), and project success results in an increased probability of further contribution.

Importantly, we demonstrate that incorporating the competing project perspective provides significantly greater explanatory power than the focal project perspective alone. This finding validates the context-specific theorizing approach (Hong et al. 2014). The relative values of extrinsic motivators across competing projects, invisible from the focal project perspective alone, are significant predictors of continued contribution, confirming that the multi-project competitive structure of OSS ecosystems introduces dynamics that single-project motivation research cannot capture. Developers do not make contribution decisions in isolation from the rest of their project portfolio, and understanding the relative characteristics of competing projects is essential for explaining continued contribution behavior.

The Relative-Motivation Effect and the Absence of Competitive Displacement

Examining contributions across competing projects, we find clear evidence for the relative-motivation effect (H1b and H2b). Developers' continued contribution to the focal project is strengthened when they are associated with competing projects in which they have lower status and that are less successful than the focal project. This confirms our intuition that relative extrinsic incentives matter, and the focal project's comparative advantage in status and success enhances developers' continued engagement with it, and shows the importance of relative, rather than absolute, status and success in shaping developer behavior across project portfolios.

However, the findings also reveal an important and interesting asymmetry. Continued contribution to the focal project is *not* diminished when developers are affiliated with competing projects that provide stronger extrinsic rewards (H1a and H2a not supported): contribution to the focal project is not adversely affected by higher status in competing projects or by affiliation with more successful competing projects than the focal project. A purely relative-motivation account would predict symmetric effects. If relative advantages of the focal project attract contributions, then relative advantages of competing projects should displace them. The absence of competitive displacement indicates that the relative-motivation logic, while correct in its positive predictions, does not fully explain how developers allocate their effort across projects. Hong et al. (2014) suggest that extending established findings to new contexts can reveal dynamics that general theories do not anticipate. Our findings illustrate this. The competing-project perspective reveals

an asymmetry not visible from the focal-project perspective alone, pointing to additional context-specific factors not captured by our extrinsic measures.

Understanding the Asymmetry: The Role of Factors Beyond Relative Extrinsic Motivation

What accounts for this asymmetry? We derived all four hypotheses from the logic that relative extrinsic motivators, developer status, and project success drive effort allocation across projects. The confirmation of H1b and H2b validates this logic. When the focal project is relatively more attractive, its motivational pull is enhanced. But the non-significant results for H1a and H2a indicate that stronger extrinsic incentives elsewhere do not pull developers away from the focal project. We posit several candidate explanations, grounded in the OSS motivation literature and in related research on participation in online communities and multi-team settings. Because our archival data do not include direct measures of these factors, the following interpretations represent theoretically informed possibilities that future research should examine.

Intrinsic Motivation as a Buffering Factor: The most prominent candidate explanation is intrinsic motivation. Prior research has established that OSS developers are motivated not only by extrinsic factors such as status and reputation but also by intrinsic motivations including enjoyment of the work, intellectual stimulation, learning for its own sake, ideological commitment to open source, and community identification (Fang and Neufeld 2009; Lakhani and Wolf 2005; Roberts et al. 2006; von Krogh et al. 2012). These intrinsic motivations are likely to be, at least in part, project-specific. Developers form attachments to particular projects through their engagement with the project's technical problems, the relationships they build with other contributors, and the sense of purpose they derive from the project's mission (Fang and Neufeld 2009; Stewart and Gosain 2006). To the extent that intrinsic motivations are anchored in a specific project, they would not be easily substituted by extrinsic rewards from competing projects, however attractive. The precise role and magnitude of intrinsic motivation in producing this asymmetry remains an open question for future research.

Social Embeddedness and Relational Commitment: A second candidate explanation draws on the concept of social embeddedness. Granovetter (1985) argues that economic actions are not isolated

decisions but are deeply embedded in ongoing social structures that generate obligations, trust, identity, and expectations. Applied to OSS development, this perspective suggests that a developer's continued contribution to a focal project depends not only on the project's relative attractiveness but also on how deeply embedded the developer is in the project's relational structure. Through repeated collaboration with maintainers and peers, participation in discussions, and the assumption of community roles (e.g., module ownership, code review responsibilities), developers accumulate relational ties and role-based expectations that are not easily transferable to another project (O'Mahony and Ferraro 2007; Fang and Neufeld 2009). These obligations and identity commitments are qualitatively different from relative extrinsic incentives as they are anchored in specific relationships within a localized community context. This complementarity between motivation and embeddedness has been suggested by von Krogh et al. (2012), who propose that developers' participation reflects a "unity-of-life" pattern shaped by ongoing social practice and commitments, not only by the comparative evaluation of incentives. Our results are consistent with this view, although a direct test would require measures of relational ties and role obligations that are not available in our data.

Project-Specific Capital and Sunk Cost Inertia: A third explanation centers on the project-specific human capital that developers accumulate through sustained engagement. As developers contribute to a project, they build familiarity with its codebase, architecture, coding conventions, tooling, and tacit knowledge about how the project operates (von Krogh et al. 2003; Singh and Phelps 2013). This cumulative, project-specific knowledge creates "sunk costs" such that the marginal cost of contributing to the focal project declines as familiarity increases, while contributing to a competing project entails new learning, coordination, and integration costs (O'Leary et al. 2011). These switching costs could make it less attractive for developers to reallocate effort away from the focal project, particularly when a developer has not invested comparable sunk costs in competing projects. This explanation is particularly relevant given the "contribution barrier," i.e., the substantial upfront investment required to become an effective contributor to any new project (Crowston et al. 2004; von Krogh et al. 2003). Developers who

have already overcome this barrier in the focal project face non-trivial costs of surmounting it again elsewhere, which may dampen responsiveness to extrinsic incentive differentials.

The Role of Time in Developing Commitment: Intrinsic motivations, social ties, and project-specific capital all develop and deepen over time through sustained engagement with the focal project. As developers accumulate tenure on a project, they are likely to form stronger affective attachments, build denser relational networks, and develop deeper technical familiarity, which could strengthen the buffering effect against competitive displacement. This temporal logic is consistent with research showing that OSS developers transition through motivational phases, moving from extrinsic "carrots" toward more intrinsic, enduring "rainbows" as they gain experience (von Krogh et al. 2012). Gerosa et al. (2021) provide supporting evidence, finding that experienced OSS developers increasingly internalize their motivations over time. This suggests that the asymmetry we observe may be especially pronounced among developers with longer tenure in the focal project, who have had more time to develop the intrinsic motivations, social ties, and project-specific capital that buffer their engagement.

In summary, while our hypotheses were derived from relative extrinsic motivation logic, the observed asymmetry indicates that this logic alone is insufficient to explain developers' effort allocation across competing projects. We acknowledge that our data do not permit us to adjudicate among these explanations or to estimate their relative contributions. Instead, the value of our findings lies in documenting a context-specific empirical pattern (Hong et al. 2014) that emerges from the multi-project structure of OSS ecosystems and is not derivable from focal-project-only motivation research. This pattern identifies the theoretical space that future research must fill with more targeted measurement and design.

Practical Implications

OSS projects have increasing societal and economic impacts (von Krogh and Spaeth 2007). The shift from volunteer-driven contributions to a landscape in which commercial organizations employ many OSS developers (Watson et al. 2008) raises new questions about managing developer engagement amid competing demands. Despite the growing presence of sponsoring organizations, the majority of OSS code

development is still undertaken by volunteer developers (von Krogh et al. 2012). Our findings offer several practical insights for project managers and maintainers.

Perhaps the most important implication is that developers' code contributions to a specific focal project cannot be considered in isolation and that OSS projects benefit from actively communicating their comparative advantages relative to competing projects, whether in terms of community recognition, project impact, or technical quality. Helping developers see that their status and the focal project's success compare favorably with alternatives may strengthen their commitment.

At the same time, our finding that relatively higher status or success of competing projects does *not* reduce focal project contributions is encouraging for project managers. It suggests that developers' engagement with a focal project is resilient to competitive pressures, likely because it is sustained by factors beyond portable extrinsic rewards. This implies that investing in the social and technical dimensions of developer experience by fostering collaborative relationships, providing mentoring, creating a welcoming community culture, and reducing barriers to contribution may be as important for retention as providing extrinsic recognition. Current OSS practice considers status achievement as a "gain" incentive allocated when certain contribution levels are reached. Our results suggest that managers may also benefit from fostering the conditions that build intrinsic commitment and social attachment, as these appear to provide a buffer against the pull of competing projects.

Methodological Implications

Methodologically, we adopt the operationalization of continued contribution using the Pareto-NBD model widely used in the marketing literature (Borle et al. 2008; Schmittlein et al. 1987). This approach is preferable to the heuristic measures used in the OSS and online community literature to date (e.g., a simple count of code commits) and can become an important tool for OSS and online community researchers in general. For example, deciding whether a developer has left the project for good after a period of inactivity ignores their historical activity pattern and their likelihood of returning. Further, it yields a dichotomous measure rather than a probabilistic one, thereby losing information by ignoring differences between more

frequently active developers and less active ones (Schmittlein et al. 1987). The Pareto-NBD approach not only considers developers' historical activity patterns but also their most recent activity. Because developer contribution levels to a project may change over time, incorporating past activity levels leads to better predictions of continued contribution. Another advantage of the Pareto-NBD approach is its ability to predict future developer activity levels (e.g., expected number of commits). Our operationalization allows OSS project managers not only to predict the future active workforce in the future and manage the project accordingly, but also to estimate the expected number of commits by summing the expectations of the current developer base (Schmittlein et al. 1987).

Limitations and Future Research

The first limitation of our study is that we measure only two extrinsic motivators (status and project success) and, as discussed above, do not directly measure intrinsic motivations or structural factors such as social embeddedness and project-specific capital. This is an inherent limitation of the archival, multi-project nature of our data. As outlined in our research agenda above, future work should pursue designs that combine archival contribution data with survey-based measures of intrinsic motivation and community identification, network-based measures of social embeddedness constructed from collaboration histories, and moderators such as technical similarity that proxy for the magnitude of switching costs. Each would provide a more direct test of the candidate explanations we have identified and help adjudicate their relative contributions to the asymmetry.

Second, the cross-sectional nature of our predictor variables constrains the causal claims we can make. Our dependent variable, *p(Alive)*, is constructed using longitudinal data (the frequency and recency of code commits over a 180-day observation window), but all predictor variables represent a snapshot in time. This design does not allow us to examine how changes in developers' status or project success over time influence the trajectory of their contributions. As we have discussed, intrinsic motivations, social ties, and project-specific capital could develop over time. Future research should collect longitudinal data on

developer status and project characteristics to model how shifts in the relative attractiveness of competing projects dynamically influence continued contribution.

Third, we consider only code commits as contributions. However, past work has shown the importance of other contributions by peripheral participants, such as code documentation, managing websites, wikis and mailing lists, and popularizing projects through positive word-of-mouth (Setia et al. 2012). While code commits indicate the software artifact being produced and are a necessary condition for project success (Crowston et al. 2006), it would be worthwhile to assess whether our results generalize to other types of contributions. This is particularly relevant because different types of contributions may be differentially responsive to extrinsic versus intrinsic motivations. For instance, non-code contributions such as mentoring, community building, and issue triage carry less extrinsic signaling value and may be more strongly driven by intrinsic motivation and relational commitment. Examining different contribution types as parallel dependent variables could help disentangle the relative influence of extrinsic incentives versus intrinsic and social factors on different facets of developer engagement.

Fourth, although Google Code was one of the most popular OSS forges, our work is strictly generalizable to platforms with similar status structures. However, this limitation may be less restrictive than it first appears, since other popular platforms (e.g., SourceForge and GitHub) also delineate developers' rights within hierarchies. The key boundary condition is that the platform must make status visible and consequential for developer behavior, a condition that most major OSS platforms satisfy. Future research could examine whether the asymmetry replicates on platforms with different governance structures, such as those with flatter hierarchies or more fluid role assignments, to test whether the strength of the relative-motivation effect depends on the formality and visibility of status distinctions.

Finally, while our sample consisted of volunteer developers working on projects not sponsored by companies, modern OSS development is now shaped by funding from for-profit companies that affect developer contributions (Sharma et al. 2022). The presence of paid contributions introduces a qualitatively different set of motivational dynamics. This presents a valuable opportunity to extend our work by examining whether the asymmetry holds in settings where developers are financially compensated.

CONCLUSION

OSS ecosystems comprise many projects that compete for volunteer developers, who typically split their attention among multiple projects. Projects that are unable to attract a critical mass of developers who make long-term, continued contributions can fail. We develop a model to examine how developers' continued contribution to a focal project is affected not only by their affiliation with the project but also by competing projects in their portfolio. Following a context-specific theorizing approach, we extend established motivation findings to this multi-project context by focusing on the relative values of two key extrinsic motivators, developer status and project success, across competing projects. Using data from Google Code, we find that developers' status in the focal project and the focal project's success drive their contribution to it. Importantly, this continued contribution is promoted when a developer has lower relative status in competing projects and when competing projects have achieved lower relative success than the focal project. Asymmetrically, continued contribution to the focal project is not diminished when a developer has higher relative status in competing projects or when these projects are more successful than the focal project. These findings extend prior work on continued contribution by highlighting the multi-project embedded nature of OSS development and the importance of considering the competing projects perspective when assessing developer engagement.

REFERENCES

Aberdour, M. 2007. "Achieving Quality in Open-Source Software," *IEEE Software* (24:1), IEEE.

Aral, S., Brynjolfsson, E., and Van Alstyne, M. 2012. "Information, Technology, and Information Worker Productivity," *Information Systems Research* (23:3-part-2), pp. 849–867. (https://doi.org/10.1287/isre.1110.0408).

Bagozzi, R. P., and Dholakia, U. M. 2006. "Open Source Software User Communities: A Study of Participation in Linux User Groups," *Management Science* (52:7), INFORMS, pp. 1099–1115.

Binstock, A. 2011. "How to Contribute to Open Source Projects," *Dr.Dobb's*. (http://www.drdobbs.com/open-source/how-to-contribute-to-open-source-project/231000080?pgno=2).

Borle, S., Singh, S. S., and Jain, D. C. 2008. "Customer Lifetime Value Measurement," *Management Science* (54:1), INFORMS, pp. 100–112.

Butler, B. S. 2001. "Membership Size, Communication Activity, and Sustainability: A Resource-Based Model of Online Social Structures," *Information Systems Research* (12:4), pp. 346–362. (https://doi.org/10.1287/isre.12.4.346.9703).

Chengalur-Smith, I., Sidorova, A., and Daniel, S. L. 2010. "Sustainability of Free/Libre Open Source Projects: A Longitudinal Study," *Journal of the Association for Information Systems* (11:11), pp. 657–683.

Crowston, K., Annabi, H., Howison, J., and Masango, C. 2004. "Effective Work Practices for Software Engineering: Free/Libre Open Source Software Development," in *Proceedings of the 2004 ACM Workshop on Interdisciplinary Software Engineering Research*, ACM, pp. 18–26.

Crowston, K., Howison, J., and Annabi, H. 2006. "Information Systems Success in Free and Open Source Software Development: Theory and Measures," *Software Process Improvement and Practice* (11:2), pp. 123–148. (https://doi.org/10.1002/spip.259).

Cummings, J. N., and Haas, M. R. 2012. "So Many Teams, so Little Time: Time Allocation Matters in Geographically Dispersed Teams," *Journal of Organizational Behavior* (33:3), pp. 316–341. (https://doi.org/10.1002/job.777).

Daniel, S., Agarwal, R., and Stewart, K. J. 2013. "The Effects of Diversity in Global, Distributed Collectives: A Study of Open Source Project Success," *Information Systems Research* (24:2), INFORMS, pp. 312–333. (https://doi.org/10.1287/isre.1120.0435).

Daniel, S., and Stewart, K. 2016. "Open Source Project Success: Resource Access, Flow, and Integration," *The Journal of Strategic Information Systems* (25:3), pp. 159–176. (https://doi.org/10.1016/j.jsis.2016.02.006).

David, P. A., and Rullani, F. 2008. "Dynamics of Innovation in an Open Source Collaboration Environment: Lurking, Laboring, and Launching FLOSS Projects on SourceForge," *Industrial and Corporate Change* (17:4), Oxford Univ Press, pp. 647–710.

Deci, E. L., and Ryan, R. 1985. "Intrinsic Motivation and Self-Determination in Human Behavior," *New York: Pantheon*.

Dziurzynski, L., Wadsworth, E., Fader, P. S., McDonnell, E., McCarthy, D., Hardie, B., Gopalakrishnan, A., Schwartz, E., and Zhang, Y. 2012. "BTYD: Implementing Buy Till You Die Models."

Fader, P. S., Hardie, B. G. S., and Lee, K. L. 2005. "'Counting Your Customers' the Easy Way: An Alternative to the Pareto/NBD Model," *Marketing Science* (24:2), INFORMS, pp. 275–284.

Fang, Y., and Neufeld, D. 2009. "Understanding Sustained Participation in Open Source Software Projects," *Journal of Management Information Systems* (25:4), ME Sharpe, pp. 9–50.

Finley, K. 2013. "Out In The Open : How To Resurrect A Dead Open Source," *Wired Magazine*.

Fitzgerald, B. 2005. *Has Open Source Software a Future*, MIT Press London, pp. 93–106.

Fitzgerald, B. 2006. "The Transformation of Open Source Software," *MIS Quarterly* (30:3), pp. 587–598.

Frey, B. S., and Jegen, R. 2001. "Motivation Crowding Theory," *Journal of Economic Surveys* (15:5), pp. 589–611. (https://doi.org/10.1111/1467-6419.00150).

Gagné, M., and Deci, E. L. 2005. "Self-Determination Theory and Work Motivation," *Journal of Organizational Behavior* (26:4), John Wiley & Sons, Ltd., pp. 331–362. (https://doi.org/10.1002/job.322).

Gerosa, M., Wiese, I., Trinkenreich, B., Link, G., Robles, G., Treude, C., Steinmacher, I., and Sarma, A. 2021. "The Shifting Sands of Motivation: Revisiting What Drives Contributors in Open Source," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, IEEE, pp. 1046–1058. (https://ieeexplore.ieee.org/abstract/document/9402044/).

Giuri, P., Ploner, M., Rullani, F., and Torrisi, S. 2010. "Skills, Division of Labor and Performance in Collective Inventions: Evidence from Open Source Software," *International Journal of Industrial*

Organization (28:1), Elsevier B.V., pp. 54–68. (https://doi.org/10.1016/j.ijindorg.2009.07.004).

Google. 2015. “Permissions in Google Code.” (https://code.google.com/p/support/wiki/Permissions).

Granovetter, M. 1985. "Economic Action and Social Structure: The Problem of Embeddedness," *American Journal of Sociology* (91:3), pp. 481–510.

Grewal, R., Lilien, G. L., and Mallapragada, G. 2006. “Location, Location, Location: How Network Embeddedness Affects Project Success in Open Source Systems,” *Management Science* (52:Icis), pp. 1043–1056. (https://doi.org/10.1287/mnsc.1060.0550).

Hann, I.-H., Roberts, J. A., and Slaughter, S. A. 2013. “All Are Not Equal: An Examination of the Economic Returns to Different Forms of Participation in Open Source Software Communities,” *Information Systems Research* (24:3), pp. 520–538. (https://doi.org/10.1287/isre.2013.0474).

Hanselman, S. 2016. “How to Contribute a Patch to an Open Source Project.” (http://www.hanselman.com/blog/ExampleHowToContributeAPatchToAnOpenSourceProjectLikeDasBlog.aspx).

Hars, A., and Ou, S. 2002. “Working for Free ? Motivations for Participating in Open-Source Projects for Free ? Motivations Projects for Participating In,” *International Journal of Electronic Commerce: Communities in the Digital Economy* (6:3), pp. 25–39.

von Hippel, E., and von Krogh, G. 2003. “Open Source Software and the ‘Private-Collective’ Innovation Model: Issues for Organization Science,” *Organization Science* (14:2), pp. 209–223.

Ho, S. Y., and Rai, A. 2017. “Continued Voluntary Participation Intention in Firm-Participating Open Source Software Projects,” *Information Systems Research* (July), Isre.2016.0687. (https://doi.org/10.1287/isre.2016.0687).

Hong, W., Chan, F. K. Y., Thong, J. Y. L., Chasalow, L. C., and Dhillon, G. 2014. “A Framework and Guidelines for Context-Specific Theorizing in Information Systems Research,” *Information Systems Research* (25:1), pp. 111–136. (https://doi.org/10.1287/isre.2013.0501).

Jensen, C., and Scacchi, W. 2007. “Role Migration and Advancement Processes in OSSD Projects: A Comparative Case Study,” *Proceedings - International Conference on Software Engineering*, pp. 364–373. (https://doi.org/10.1109/ICSE.2007.74).

Joyce, E., and Kraut, R. E. 2006. “Predicting Continued Participation in Newsgroups,” *Journal of Computer-Mediated Communication* (11:3), Wiley Online Library, pp. 723–747.

Karlin, S., and Taylor, H. M. 1975. *A First Course in Stochastic Processes*, Academic Press, New York.

Ke, W., and Zhang, P. 2010. “The Effects of Extrinsic Motivations and Satisfaction in Open Source Software Development,” *Journal of the Association for Information Systems* (11:12), pp. 784–808.

Krishna, V., Shrestha, Y. R., and von Krogh, G. 2024. “‘The Double-Edged Sword of Inflated Help’: Unravelling the Motivation Crowding in Community Question-Answering Platforms,” *PLoS ONE* (19:3 March), pp. 1–20. (https://doi.org/10.1371/journal.pone.0297627).

Krishnamurthy, S. 2002. “Cave or Community?: An Empirical Examination of 100 Mature Open Source Projects,” *First Monday* (7:6).

Krishnamurthy, S. 2006. “On the Intrinsic and Extrinsic Motivation of Free/Libre/Open Source (FLOSS) Developers,” *Knowledge, Technology & Policy* (18:4), pp. 17–39. (https://doi.org/10.1007/s12130-006-1002-x).

Krishnamurthy, S., Ou, S., and Tripathi, A. K. 2014. “Acceptance of Monetary Rewards in Open Source Software Development,” *Research Policy* (43:4), Elsevier B.V., pp. 632–644.

(https://doi.org/10.1016/j.respol.2013.10.007).

von Krogh, G., Haefliger, S., Spaeth, S., and Wallin, M. W. 2012. "Carrots and Rainbows: Motivation and Social Practice in Open Source Software Development," *MIS Quarterly* (36:2), pp. 649–676.

von Krogh, G., and Spaeth, S. 2007. "The Open Source Software Phenomenon: Characteristics That Promote Research," *The Journal of Strategic Information Systems* (16:3), pp. 236–253. (https://doi.org/10.1016/j.jsis.2007.06.001).

von Krogh, G., Spaeth, S., and Lakhani, K. 2003. "Community, Joining, and Specialization in Open Source Software Innovation: A Case Study," *Research Policy* (32:7), pp. 1217–1241.

Lakhani, K., and Wolf, R. 2005. "Why Hackers Do What They Do: Understanding Motivation and Effort in Free/Open Source Software Projects," in *Perspectives on Free and Open Source Software*, J. Feller, B. Fitzgerald, S. Hissam, and K. Lakhani (eds.), MIT Press.

Lampel, J., and Bhalla, A. 2007. "The Role of Status Seeking in Online Communities: Giving the Gift of Experience," *Journal of Computer-Mediated Communication* (12:2), pp. 100–121. (https://doi.org/10.1111/j.1083-6101.2007.00332.x).

Lerner, J., and Tirole, J. 2002. "Some Simple Economics of Open Source," *The Journal of Industrial Economics* (L:2), Wiley Online Library, pp. 197–234. (http://www.blackwellpublishing.com/journal.asp?ref=0022-1821).

Lester, A. 2012. "14 Ways to Contribute to Open Source without Being a Programming Genius or a Rock Star," *Smartbear Blog*. (http://blog.smartbear.com/programming/14-ways-to-contribute-to-open-source-without-being-a-programming-genius-or-a-rock-star/).

Levina, N., and Arriaga, M. 2014. "Distinction and Status Production on User-Generated Content Platforms: Using Bourdieu's Theory of Cultural Production to Understand Social Dynamics in Online Fields," *Information Systems Research* (25:3), pp. 468–488. (https://doi.org/10.1287/isre.2014.0535).

Maas, C. J. M., and Hox, J. J. 2004. "Robustness Issues in Multilevel Regression Analysis," *Statistica Neerlandica* (58:2), Wiley Online Library, pp. 127–137.

Marsan, J., Paré, G., and Beaudry, A. 2012. "Adoption of Open Source Software in Organizations: A Socio-Cognitive Perspective," *The Journal of Strategic Information Systems* (21:4), Elsevier B.V., pp. 257–273. (https://doi.org/10.1016/j.jsis.2012.05.004).

Mithas, S., and Krishnan, M. S. 2009. "From Association to Causation via a Potential Outcomes Approach," *Information Systems Research* (20:2), pp. 295–313. (https://doi.org/10.1287/isre.1080.0184).

Mockus, A., Fielding, R. T., and Herbsleb, J. D. 2002. "Two Case Studies of Open Source Software Development: Apache and Mozilla," *ACM Transactions on Software Engineering and Methodology (TOSEM)* (11:3), pp. 309–346. (https://doi.org/10.1145/567793.567795).

Moon, J. Y., and Sproull, L. 2000. "Essence of Distributed Work: The Case of the Linux Kernel," *First Monday* (5:11).

Morgan, L., and Finnegan, P. 2014. "Beyond Free Software: An Exploration of the Business Value of Strategic Open Source," *The Journal of Strategic Information Systems* (23:3), Elsevier B.V., pp. 226–238. (https://doi.org/10.1016/j.jsis.2014.07.001).

O'Leary, M. B., Mortensen, M., and Woolley, A. W. 2011. "Multiple Team Membership: A Theoretical Model of Its Effects on Productivity and Learning for Individuals and Teams.," *Academy of Management Review* (36:3), pp. 461–478. (https://doi.org/10.5465/AMR.2011.61031807).

O'Mahony, S., and Ferraro, F. 2007. "The Emergence of Governance in an Open Source Community," *Academy of Management Journal* (50:5), pp. 1079–1106. (https://doi.org/10.5465/amj.2007.27169153).

Raudenbush, S. W., and Bryk, A. S. 2002. *Hierarchical Linear Models: Applications and Data Analysis Methods*, (Vol. 1), Sage.

Riehle, D., Riemer, P., Kolassa, C., and Schmidt, M. 2014. "Paid vs. Volunteer Work in Open Source," in *System Sciences (HICSS), 2014 47th Hawaii International Conference On*, IEEE, pp. 3286–3295.

Roberts, J. a., Hann, I.-H. H., and Slaughter, S. a. 2006. "Understanding the Motivations, Participation, and Performance of Open Source Software Developers: A Longitudinal Study of the Apache Projects," *Management Science* (52:7), pp. 984–999. (https://doi.org/10.1287/mnsc.1060.0554).

Santos, C., Kuk, G., Kon, F., and Pearson, J. 2013. "The Attraction of Contributors in Free and Open Source Software Projects," *The Journal of Strategic Information Systems* (22:1), Elsevier, pp. 26–45. (https://doi.org/10.1016/j.jsis.2012.07.004).

Schmittlein, D. C., Morrison, D. G., and Colombo, R. 1987. "Counting Your Customers: Who-Are They and What Will They Do Next?," *Management Science* (33:1), INFORMS, pp. 1–24.

Setia, P., Rajagopalan, B., Sambamurthy, V., and Calantone, R. 2012. "How Peripheral Developers Contribute to Open-Source Software Development," *Information Systems Research* (23:1), INFORMS, pp. 144–163.

Sharma, P.N., Daniel, S.L., Chung, T.R., and Grover, V. 2022. "A Motivation-hygiene Model of Open Source Software Code Contribution and Growth," *Journal of the Association for Information Systems* (23:1), pp. 165–195.

Singh, J., Hansen, M. T., and Podolny, J. M. 2010. "The World Is Not Small for Everyone: Inequity in Searching for Knowledge in Organizations," *Management Science* (56:9), pp. 1415–1438. (https://doi.org/10.1287/mnsc.1100.1201).

Singh, P. V., and Phelps, C. 2013. "Networks, Social Influence, and the Choice Among Competing Innovations: Insights from Open Source Software Licenses," *Information Systems Research* (24:3), pp. 539–560. (https://doi.org/10.1287/isre.1120.0449).

Singh, P. V., Tan, Y., and Youn, N. 2011. "A Hidden Markov Model of Developer Learning Dynamics in Open Source Software Projects," *Information Systems Research* (22:4), INFORMS, pp. 790–807. (http://dx.doi.org/10.1287/isre.1100.0308).

Snijders, T., and Bosker, R. 2012. *Multilevel Analysis: An Introduction to Basic and Advanced Multilevel Modeling*, Sage.

Snyder, C. R., Lassegard, M., and Ford, C. E. 1986. "Distancing after Group Success and Failure: Basking in Reflected Glory and Cutting off Reflected Failure.," *Journal of Personality and Social Psychology* (51:2), pp. 382–388. (https://doi.org/10.1037/0022-3514.51.2.382).

Spaeth, S., von Krogh, G., and He, F. 2014. "Research Note—Perceived Firm Attributes and Intrinsic Motivation in Sponsored Open Source Software Projects," *Information Systems Research* (26:1), Informs, pp. 224–237.

Stewart, K. J., Ammeter, A. P., and Maruping, L. M. 2006. "Impacts of License Choice and Organizational Sponsorship on User Interest and Development Activity in Open Source Software Projects," *Information Systems Research* (17:2), pp. 126–144. (https://doi.org/10.1287/isre.1060.0082).

Stewart, K. J., and Gosain, S. 2006. "The Impact of Ideology on Effectiveness in Open Source Software Development Teams," *MIS Quarterly* (30:April 2005), pp. 291–314.

(https://doi.org/10.2307/25148732).

Sutanto, J., Jiang, Q., and Tan, C.-H. 2021. "The Contingent Role of Interproject Connectedness in Cultivating Open Source Software Projects," *The Journal of Strategic Information Systems* (30:1), Elsevier, p. 101598.

Wageman, R., Gardner, H., and Mortensen, M. 2012. "The Changing Ecology of Teams: New Directions for Teams Research," *Journal of Organizational Behavior* (33:3), pp. 301–315. (https://doi.org/10.1002/job.1775).

Watson, R. T., Boudreau, M.-C., York, P. T., Greiner, M. E., and Wynn, D. Jr. 2008. "The Business of Open Source," *Commun. ACM* (51:4), pp. 41–46. (https://doi.org/10.1145/1330311.1330321).

Weng, Q., and Soh, F. 2023. "The Influence of Project Initiators' Person-to-Person Followership on Project Popularity in Open Source Communities: The Role of Reach and Importance," *The Journal of Strategic Information Systems* (32:2), Elsevier, p. 101771.

Wooldridge, J. M. (2016). *Introductory econometrics a modern approach*. South-Western Cengage Learning.

Wu, C. G., Gerlach, J. H., and Young, C. E. 2007. "An Empirical Analysis of Open Source Software Developers' Motivations and Continuance Intentions," *Information and Management* (44:3), pp. 253–262. (https://doi.org/10.1016/j.im.2006.12.006).

Xu, B., Jones, D. R., and Shao, B. 2009. "Volunteers' Involvement in Online Community Based Software Development," *Information & Management* (46:3), Elsevier, pp. 151–158.

Zhang, C., Hahn, J., and De, P. 2013. "Continued Participation in Online Innovation Communities: Does Community Response Matter Equally for Everyone?," *Information Systems Research* (24:4), pp. 1112–1130. (https://doi.org/10.1287/isre.2013.0485).

Zhu, H., Kraut, R. E., and Kittur, A. 2014. "The Impact of Membership Overlap on the Survival of Online Communities," *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, pp. 281–290. (https://doi.org/10.1145/2556288.2557213).

ONLINE APPENDIX A:
FURTHER NOTES ON ESTIMATION METHODOLOGY

Our approach for estimating our dependent variable (continued code contribution; *p(Alive)*) is adapted from Schmittlein et al. (1987), who analyzed customer lifetime values in non-contractual settings. Their model assumes that customers buy at a steady (albeit stochastic) rate for a period of time and then become inactive. A similar modeling structure can be employed here, because a developer can always return to a project after the observation period has ended. Any arbitrary window of time selected to study this issue will result in right-censored measurement of dropout (e.g. Joyce and Kraut 2006; Zhang et al. 2013). Using both a Pareto (exponential-gamma mixture) timing model and an NBD (Poisson-gamma mixture) activity level model, our approach allows us to explicitly deal with the issue of right-censored

data by predicting a *probability* of continued contribution (Fader et al. 2005).

Under this framework, a developer's activity on the code for a specific project (i.e., a code commit) can be considered an "event". A developer who is active and still contributing is considered "*Alive*," whereas a developer who is inactive is considered "*Dead*," with $p(Alive) = 1 - p(Dead)$. Using an event window running for time $t = 0$ to T, we can observe the total number of code contributions (x) the developer makes to the project, with the last contribution coming at time t_x, $0 < t_x \leq T$. The value x represents the *frequency* of contribution activity, whereas t_x reflects the *recency* of that activity. The Pareto-NBD approach can then be used to calculate the conditional probability that the developer is still alive after the observation period ending at time T, $p(Alive|(x, t_x, T))$, subject to the following assumptions described by Schmittlein et al. (1987):

(1) While alive, a developer *"i"* makes stochastic contributions to the code according to a Poisson process with a rate λ_i. The Poisson process implies exponentially distributed commit times, with a resulting "lack of memory" property. This is a reasonable assumption in the OSS context, since the development of code is not bound by any regular schedule, and each developer is likely to have a unique but constant probability to contribute code (i.e. their own "pace" of contribution).

(2) Each developer remains alive for a lifetime that has an exponentially distributed duration, with death rate μ_i. The events that could trigger "death" (such as a new career, lifestyle change, etc.) are assumed to arrive in a Poisson manner. The arrival of all possible death (dropout) events is the superposition of the individual events, and is best approximated as a Poisson process (Karlin and Taylor 1975). This assumption also appears reasonable in the OSS context.

(3) The activity rate λ_i and the death rate μ_i differ by developer i, and these rates are distributed according to different gamma distributions in the population. Further, λ_i and μ_i are distributed independently of each other. It is reasonable to assume that different developers have different contribution activity and death rates. Some developers are more active than others (leading to differences in λ_i), and some developers may become disenchanted with a project sooner than others (leading to differences in μ_i). The gamma distribution is flexible enough that it can capture most

reasonable empirical distributions.

(4) Finally, there is no a-priori reason to expect either a positive or a negative correlation between λ_i and μ_i, suggesting that the two are uncorrelated with one another.

Given the above assumptions, along with information about the developer's frequency and recency of contributions to an OSS project, it is possible to calculate the developer's conditional probability of continued contribution. Each OSS project maintains a code repository that contains the complete history of its development, including each individual developer's activity history. In line with previous research (e.g. Joyce and Kraut 2006), in the main analysis we observed the developers over a 180 day observation period starting from when the independent variables were measured. Thus, there is a built-in time lag between the assessment of the IVs and the calculation of the probabilistic dependent measure. To improve confidence in our findings we also observed the developers over a shorter (90 day) period.[13] The dependent variable (*pAlive(180)*) was calculated for each developer – project pair in our sample using the "Buy-Till-You-Die" (BTYD) package in R (Dziurzynski et al. 2012).[14] In general, to be evaluated as active, developers with histories of frequent contributions have to not only maintain their contribution levels but also to remain active throughout the event window. Conversely, historically lethargic contributors may attain a higher predicted chance of being alive (active) if they have more recently been active because the model assumes a slower individual activity rate.

ONLINE APPENDIX B: ROBUSTNESS CHECKS

Approaches to Address Endogeneity Concerns

[13] We present these results later as part of our robustness checks section and note that our results are generalizable to this period as well (model G in Appendix Table B2).

[14] Since the outcome variable is a probability measure, it is recommended that the arcsine transformation (i.e. $sin^{-1}(\sqrt{p(Alive)})$) be performed in order to normalize the distribution (Hogg and Craig, 1995). Our results remained unchanged when using the transformed version of the DVs. Hence we report our results using the untransformed versions for ease of interpretation.

An inherent interest in the focal project could be a hidden variable influencing both the developer's status in the focal project and their number of code commits. For instance, developers with higher interest may be more productive and may also self-select into (or be awarded) higher roles. This raises the possibility that our focal project variables, *Contributor* and *Committer*, could be endogenous. Although these variables are not the main focus of our hypotheses and are included only as controls (for the focal project perspective), we proactively addressed endogeneity concerns through four specific strategies outlined below:

1. Dependent Variable Construction to Mitigate Endogeneity:

The construction of our dependent variable *p(Alive)* using the Pareto/NBD model mitigates endogeneity concerns. The reason for this is that Pareto/NBD model jointly considers the frequency (number of code commits in the 180-day observation window) and the recency (the last day of code commit activity within the 180-day window) of individual developers' contributions. This dual consideration helps mitigate self-selection bias for two key reasons:

- **Differentiation from Heuristic Approaches:** Unlike heuristic methods traditionally used to model turnover, the Pareto/NBD model assumes varying activity and attrition rates among developers. This assumption is critical because it accounts for different latent propensities towards OSS contributions, addressing potential hidden variables such as inherent interest in OSS.
- **Frequency and Recency Interaction:** Developers with frequent contributions must maintain their activity levels throughout the observation window to achieve a high *p(Alive)*. Conversely, developers with historically lower contribution levels might still have a higher predicted chance of being active if their contributions are more recent. This interaction between frequency and recency reduces the bias that could arise from self-selection based solely on contribution counts.

As described in Appendix A, the Pareto/NBD model assumes that developers' contribution rates (λ_i) and attrition rates (μ_i) are exponentially distributed, independent from one another, and importantly, varying across developers. The first parameter estimates an individual developer's contribution rate (i.e.,

involvement), whereas the second estimates the developer's personal predisposition to become inactive on any given project. These values are not observed directly, but they allow us to at least partially control for the individual-specific effects of developer involvement and attrition. Thus, the inherent structure of the Pareto/NBD model accounts for unobserved heterogeneity across developers.

Our choice of the Pareto/NBD model was made on the basis of a substantial body of literature demonstrating its effectiveness in accounting for heterogeneity in individual behavior over time (Fader et al. 2005; Schmittlein, Morrison, & Colombo, 1987). By considering both the timing and quantity of contributions, the model addresses potential biases arising from unobserved heterogeneity and self-selection, common concerns in longitudinal data analysis (Wooldridge, 2016).

The individual Pareto/NBD parameter values describe an individual developer's average rate of contribution. This can be compared to their rate of contribution in a focal project. Using this information, an inference can be made regarding whether or not they are still contributing to (involved with) a particular project. For example, developers with frequent contributions must maintain their activity levels throughout the observation window to achieve a high value of *p(Alive)*. A sudden drop in contribution volume hints at potential project discontinuation by the developer. Conversely, developers with historically lower contribution levels might have a higher predicted chance of being active if their contributions are more recent. This interaction between frequency and recency reduces the bias that could arise from self-selection based solely on overall contribution counts.

Illustrative examples demonstrate how the Pareto/NBD model differentiates developers based on their activity patterns:

Developer	**Frequency**	**Recency**	**Days Since Last Activity**	***p(Alive)***
1	10	69	111	0.01
2	4	162	18	0.93
3	8	87	93	0.03
4	8	167	13	0.94
5	5	180	0	1

6	57	180	0	1

Note: Frequency is the total number of code commits made in the 180-day observation window. Recency is the day on which their last code commit was made within the 180-day observation window.

Consider these examples:

- **Developer 1 vs. Developer 2:** Despite Developer 1 having a higher frequency of commits (10) than Developer 2 (4), their last contribution occurred much earlier (day 69 vs. day 162). Thus, Developer 1 contributed more intensely, but all of their contribution was concentrated within a shorter initial period followed by a long period of inactivity. In contrast, Developer 2 made fewer contributions overall, but contributed over a more spread out duration including with more recent activity. The model identifies Developer 1 as inactive due to the extended period since their last activity.
- **Developers 3 and 4:** Both have the same frequency (8 commits), but Developer 4's more spread-out and recent activity results in a significantly higher *p(Alive)*.
- **Developers 5 and 6:** Both are considered "alive" with a probability of 1 due to their very recent activity, highlighting the model's emphasis on recency alongside frequency.

These examples highlight how the Pareto/NBD model attenuates endogeneity bias by incorporating both frequency and recency, thereby adjusting for different levels of interest in OSS contributions. Developers with higher status or more contributions do not necessarily have a higher probability of being active unless their recent activity supports it.

2. Wu-Hausman and Sargan Specification Tests using 2SLS

To further assess whether our choice of dependent variable construction helped attenuate the endogeneity bias possibly affecting the *Contributor* and *Committer* variables, we conducted a Wu-Hausman specification test and the Sargan test by comparing the OLS and 2SLS IV estimates using the *AER* package in R. We utilized three variables as instruments: (1) natural log of the total number of stars

earned by all projects that a developer is associated with, (2) natural log of the total number of software releases by all projects that a developer is associated with, and (3) natural log of the total number of issues resolved by all projects that a developer is associated with. The logic is that developers who are associated with multiple successful projects in their portfolio are more likely to assume higher status in the focal project.

We first assessed the hypothesis of weak instruments. The two F-tests were both significant at the $p = 0.001$ level, thereby rejecting the null hypothesis of weak instruments. Subsequently, we used two specification tests to compare the OLS estimates with the 2SLS instrumental variable estimates to determine if IV estimation is necessary (Wooldridge, 2016). Both, the Wu-Hausman test ($p=0.19$) and the Sargan test ($p=0.060$) fail to reject the null hypothesis that the OLS and 2SLS IV estimates are equivalent. These results suggest that endogeneity due to self-selection is not an issue in our analysis.

3. Propensity Score Matching

As a third precaution to ensure that endogeneity is not affecting the *Contributor* and *Committer* variables, we conducted propensity score matching (PSM). PSM is a popular approach in economics and statistics, and unlike the instrument-variable approach it does not assume constant treatment effect for all individuals (Mithas and Krishnan 2009). PSM overcomes this problem by identifying and matching similar cases over *observed* covariates and comparing only matched cases over the dependent variable of interest. We conducted two separate analyses by matching developers over their status distributions in competing projects using the nearest-neighbor matching algorithm. First, owners were matched with similar committers and the HLM model was re-estimated using only the matched subjects. The focus of interest in this analysis was the parameter estimate of $\beta_{1_PSM} = -.138, p < .001$, which is close to our original estimate of $\beta_1 = -.123, p < .001$. The second analysis focused on matching owners and contributors and the new estimate for $\beta_{2_PSM} = -.189, p < .001$, is also close to the original estimate of $\beta_2 = -.170, p < .001$. These analyses support the robustness of our original estimates.

4. Addressing Reverse Causality Using Time-Lagged DV

A final endogeneity related concern is the issue of possible reverse causality between focal project success and developers' contribution levels to the focal project. That is, developers' code contribution levels can also enhance project success (our model assumes the reverse, i.e., that focal project success leads to developers' continued contributions). To separate the two effects, we utilized a time-lag between when success of the focal project was measured versus when developers' code commits were measured. As noted in the manuscript, project success was assessed ***prior*** to the 180-day code contribution observation window. Thus, our results show the unidirectional effect of project success on subsequent code contribution activity, and not vice-versa.

Alternate operationalizations of competing project relative status and relative success

Relative status in competing projects (Including EqualStatus):

We had operationalized developers' relative status by using the count of competing projects where developers have higher status or lower status However, in some cases developers had an equal status in a competing project. In particular, owners of focal projects cannot have a higher status in other competing projects, but they may have equal status as owners in those projects (vice-versa for contributors). To analyze whether such competing projects also affected developers' contribution to their focal project, we incorporated an additional variable *EqualStatus* in our model, which is a count of competing projects where developers have the same status as in the focal project. Model A in table B1 shows that our results remain unchanged.

Relative success of competing projects:

We also analyzed the effect of alternate operationalizations for the *MoreDownloads* and *LessDownloads* variables by using smaller and larger cutoffs (Models B-D in Table B1). Model B uses a strictly higher (lower) number of downloads as the cut off, such that *MoreDownloads* (*LessDownloads*) represents the number of competing projects with higher (lower) number of downloads than the focal project. Model C (10 download difference) and D (100 download difference) represent lower and higher cutoff versions of our original operationalization. Our conclusions remain unchanged when using these

alternate operationalizations of the relative success of competing projects.

Additional focal and competing project-level variables

We evaluated robustness in the presence of additional focal and competing project variables. First, at the focal project level, the number of software releases is an indicator of the project's adherence to the "release early, release often" ideology to manage timely software development, and serves as an indicator of its vitality. Focal projects with frequent releases reflect the presence of a highly active developer team. Further, it may not just be the availability and extent of documentation that matters for code development, but rather its quality. Good quality documentation can help developers understand the software system better, thereby, making it easier for them to contribute code (Crowston, Howison, and Annabi, 2006). On Google Code, the number of stars received indicates how helpful the community finds the project documentation to be and serves as a proxy of its quality. Thus, we include *NumReleases* and *WikiStars* as two additional variables at the focal project level to control for these effects.

Second, at the competing project level, it is possible that certain developers tend to work more on relatively newer competing projects that may draw their attention away from the focal project. We included the average age of competing projects to control for this effect (*AverageCompetingProjectAge*). Working on more responsive competing projects could also affect developers' contributions to the focal project, thus we included the average of the proportion of issues resolved across their competing projects as an additional control (*AverageProportionIssuesResolved*). Model E (Table B1), which incorporates these effects simultaneously, provides strong evidence of the robustness of our results against these alternative explanations.

Table B1
Alternate Operationalizations & Additional Variables

Fixed Effects	**Model A: Including *EqualStatus***	**Model B: Strictly Higher/Lower Downloads**	**Model C: Difference of 10 or more Downloads**	**Model D: Difference of 100 or more Downloads**	**Model E: Additional Variables**
Intercept	0.249*** (0.014)	0.255*** (0.014)	0.254*** (0.017)	0.251*** (0.014)	0.254*** (0.014)

Controls					
Centrality	-0.000 (0.000)	-0.000 (0.000)	-0.000 (0.000)	-0.000 (0.000)	-0.000 (0.000)
Stars	0.005 (0.017)	0.003 (0.017)	0.000 (0.017)	0.008 (0.017)	-0.010 (0.019)
CompetingActivity	0.001** (0.000)	0.001** (0.000)	0.001** (0.000)	0.001* (0.000)	0.001** (0.000)
Age	-0.004 (0.003)	-0.004 (0.003)	-0.004 (0.003)	-0.003 (0.003)	-0.002 (0.003)
PermissiveLicense	-0.028** (0.013)	-0.028** (0.013)	-0.027** (0.013)	-0.028** (0.013)	-0.031** (0.014)
OtherLicense	0.025 (0.047)	0.022 (0.046)	0.023 (0.046)	0.032 (0.048)	0.038 (0.047)
NumDevelopers	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)
WikiPages	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)
ProportionIssuesResolved	0.045** (0.021)	0.046** (0.021)	0.045** (0.021)	0.043** (0.020)	0.036* (0.020)
EqualStatus	0.002 (0.002)	-	-	-	-
NumReleases	-	-	-	-	0.000 (0.000)
WikiStars	-	-	-	-	0.000 (0.000)
AverageCompetingProjectAge	-	-	-	-	0.001 (0.004)
AverageProportionIssuesResolved	-	-	-	-	0.048* (0.025)
Committer	-0.122*** (0.016)	-0.123*** (0.016)	-0.122*** (0.016)	-0.124*** (0.016)	-0.120*** (0.016)
Contributor	-0.169*** (0.018)	-0.170*** (0.018)	-0.168*** (0.018)	-0.171*** (0.018)	-0.168*** (0.018)
Downloads	0.010** (0.004)	0.010** (0.004)	0.010** (0.004)	0.010** (0.004)	0.008** (0.004)
Hypothesized Effects					
HigherStatus	0.000 (0.004)	-0.000 (0.003)	-0.001 (0.003)	-0.000 (0.003)	-0.003 (0.004)
LowerStatus	0.029*** (0.009)	0.028*** (0.009)	0.028*** (0.009)	0.029*** (0.009)	0.022** (0.009)
MoreDownloads	-0.006 (0.007)	-0.002 (0.006)	-0.002 (0.007)	-0.004 (0.006)	-0.007 (0.007)
LessDownloads	0.004* (0.002)	0.004** (0.002)	0.005** (0.002)	0.004** (0.002)	0.005** (0.002)

Alternate model specifications

We also assessed the robustness of our results by utilizing several alternate model specifications. First, while full maximum likelihood (FML) estimation allowed us to compare nested models based on deviance statistics, an alternative estimation method, residual maximum likelihood (REML), often produces less biased estimates and more reliable standard errors than FML, especially when normality cannot be assumed (Maas and Hox 2004; Snijders and Bosker 2012). The REML estimates (Model F; Table B2) are consistent with our original FML estimates (c.f. Table 6 in the paper). More importantly, the magnitudes of robust standard errors in both cases are very similar, providing additional support for the robustness of estimates.

Second, as previously discussed, we tested our hypotheses using a 180-day observation period for developer contributions – a time choice in line with past research (Joyce and Kraut 2006). However, this length is somewhat arbitrary. In order to check the validity of our results, we re-estimated our final model using a 90-day observation period, a subset of our original observation window (Model G; Table B2). The pattern of results is generally consistent with our findings for the 180-day period, with a few exceptions. The effect of *LowerStatus* was not significant in the 90-day period; in contrast the effect of *MoreDownloads* was significant at the 0.10 level. Finally, while the focal project success (*Downloads*) has a positive effect on continued contribution across the full (180 days) observation window, it was not significant in the shorter term (i.e. 90 days). A potential reason for this difference might be that focal project success represents an external validation of its value, the interest it generates, and thus may have a more distal impact.

Third, because contributors were a minority in our sample (7.4%), we also examined the stability of our results by excluding them from the analysis and comparing only owners and committers. These results (Model H: Table B2) validate our earlier findings.

Fourth, while we note the empirical strengths and practical benefits associated with the Pareto-NBD measure for continued contribution, we also tested whether our results were consistent with an alternate operationalization using the count of code commits made over a period of 180 days, which has

been used in past research (e.g., Joyce and Kraut 2006). Model I in Table B2 shows that our hypothesized effects and results are robust when using this alternate operationalization.

Fifth, we took two steps to address concerns regarding the skewed distributions of some of the variables.[15] In particular, five variables in our model have high standard deviations compared to their means (*Centrality, WikiPages, CompetingProjectActivity, HigherStatus,* and *LessDownloads*). First, in our main analysis we reported the Huber-White robust standard errors that are consistent under violations of HLM assumptions (non-normality and outliers), for all model parameters (Raudenbush & Bryk, 2002). The model-based standard errors (not reported) and the robust standard errors were similar in magnitude suggesting that model misspecification and violations of assumption were not a concern for our analysis (Raudenbush et al. 2011). Second, to ensure that our assumptions regarding the first step were correct, we conducted outlier analysis by rerunning our main model by utilizing the winsorized versions of the concerned variables. To do so, we replaced top 5% values of the variable by its 95th percentile value. This procedure resulted in considerable reduction in the standard deviations as follows:

Winsorized Variable	**Mean**	**SD**	**Min.**	**Max.**
Centrality	27.54	25.67	2.00	94.00
WikiPages	7.49	10.38	0.00	39.00
CompetingProjectActivity	0.57	1.90	0.00	8.00
HigherStatus	0.50	1.07	0.00	4.00
LessDownloads	1.28	1.96	0.00	7.00

The results reported in Model J (Table B2) using the winsorized variables show that our conclusions are robust against the skewed distribution of these variables.

Finally, it is possible that developers who have higher (or lower) status in secondary projects may have different probabilities to continue contributing to a focal project based on higher (or lower) success of their secondary projects.[16] To assess the combined effect of developers' relative statuses in competing projects and the relative success of their competing projects on their continued contribution to a focal

[15] We thank the anonymous reviewer for suggesting this addition.
[16] We thank the anonymous reviewer for suggesting this analysis.

project, we reran our model by including four interaction terms in our model (*HigherStatus x MoreDownloads, HigherStatus x LessDownloads, LowerStatus x MoreDownloads,* and *LowerStatus x LessDownloads*). The results presented in Model K (Table B2) show that the interaction terms are either statistically not significant and/or have very low effect sizes, thus ruling out any appreciable combined effects. More importantly, our main conclusions remain robust against the inclusion of the four interaction terms.

Table B2
Alternate Model Specifications

Fixed Effects	**Model F: REML Estimates**	**Model G: 90 Day Observation Window**	**Model H: Excluding Contributors**	**Model I: Alternate DV - Count of Code Commits**	**Model J: Winsorized Variables**	**Model K: Interaction Effects**
Intercept	0.249*** (0.014)	0.287*** (0.012)	0.222*** (0.012)	5.077*** (0.511)	0.258*** (0.015)	0.262*** (0.015)
Controls						
Centrality	-0.000 (0.000)	-0.000 (0.000)	-0.000 (0.000)	-0.022** (0.007)	-0.001 (0.000)	-0.000** (0.000)
Stars	-0.005 (0.017)	0.056 (0.022)	0.006 (0.017)	1.655** (0.686)	0.003 (0.020)	0.005 (0.019)
CompetingActivity	0.001** (0.000)	0.001 (0.000)	0.001** (0.000)	0.034 (0.028)	0.013** (0.003)	0.001** (0.000)
Age	-0.004 (0.003)	-0.002 (0.003)	-0.004 (0.003)	-0.114 (0.105)	-0.004 (0.003)	-0.004 (0.003)
PermissiveLicense	-0.030** (0.016)	-0.034** (0.011)	-0.022 (0.013)	-0.881** (0.414)	-0.023* (0.013)	-0.023* (0.013)
OtherLicense	0.024 (0.047)	0.000 (0.029)	0.035 (0.048)	1.606 (1.537)	0.042 (0.054)	0.002 (0.039)
NumDevelopers	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.021 (0.024)	0.000 (0.000)	0.000 (0.000)
WikiPages	0.000 (0.000)	0.000 (0.000)	0.000 (0.000)	0.005 (0.008)	0.000 (0.000)	-0.000 (0.000)
ProportionIssuesResolved	0.044** (0.021)	0.055** (0.017)	0.022 (0.020)	1.438** (0.688)	0.043** (0.020)	0.051** (0.021)
Committer	-0.124*** (0.016)	-0.086*** (0.013)	-0.081*** (0.013)	-3.664*** (0.554)	-0.126*** (0.018)	-0.129*** (0.016)
Contributor	-0.171*** (0.018)	-0.142*** (0.015)	-	-5.124*** (0.696)	-0.165*** (0.020)	-0.173*** (0.019)
Downloads	0.010** (0.004)	0.002 (0.003)	0.010** (0.004)	-0.105 (0.126)	0.007* (0.004)	0.010** (0.004)
Hypothesized Effects						

HigherStatus	-0.001 (0.006)	-0.003 (0.003)	-0.007* (0.003)	0.090 (0.101)	-0.007 (0.007)	-0.001 (0.004)
LowerStatus	0.026** (0.010)	0.003 (0.005)	0.038*** (0.008)	1.133*** (0.370)	0.028*** (0.008)	0.026** (0.011)
MoreDownloads	-0.005 (0.006)	-0.008* (0.005)	-0.003 (0.006)	-0.175 (0.152)	-0.000 (0.007)	0.005 (0.009)
LessDownloads	0.004* (0.002)	0.003** (0.001)	0.009*** (0.002)	0.163** (0.065)	0.010** (0.005)	0.009*** (0.003)
Interaction Effects (Post Hoc)						
HigherStatus x MoreDownloads						0.000 (0.001)
HigherStatus x LessDownloads						-0.000* (0.000)
LowerStatus x MoreDownloads						-0.003 (0.003)
LowerStatus x LessDownloads						0.000** (0.000)